



深度生成模型

Deep Generative Model



01 引言(Introduction)

02 变分自编码器(Variational AutoEncoder)

03 生成对抗网络(Generative Adversarial Network)

04 小结与资源 (Take Home Message and Resource)

PART ONE

引言

Introduction

➤ 猜一猜：哪一张图像是相机拍摄的，哪一张是用算法生成的？



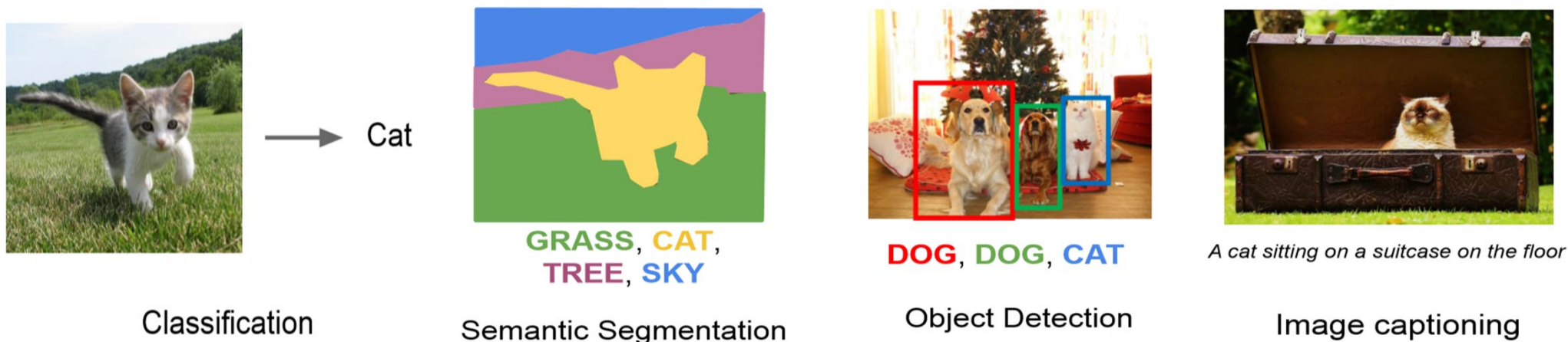
二者均是由NVIDIA提出的一种深度生成模型产生的！

Karras *et al.*, A Style-Based Generator Architecture for Generative Adversarial Networks, CVPR2019.

■ 回顾：有监督学习 vs 无监督学习

➤ 监督学习 (Supervised Learning)

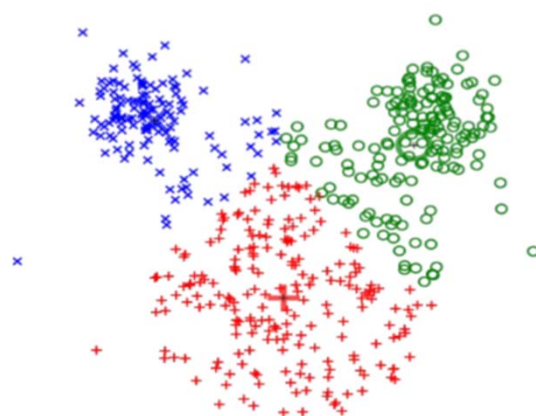
- 有标签数据: (x, y) , y 为标签
- 目标: 学习映射函数 $x \rightarrow y$
- 例: 图像分类、语义分割、目标检测、自动图像描述



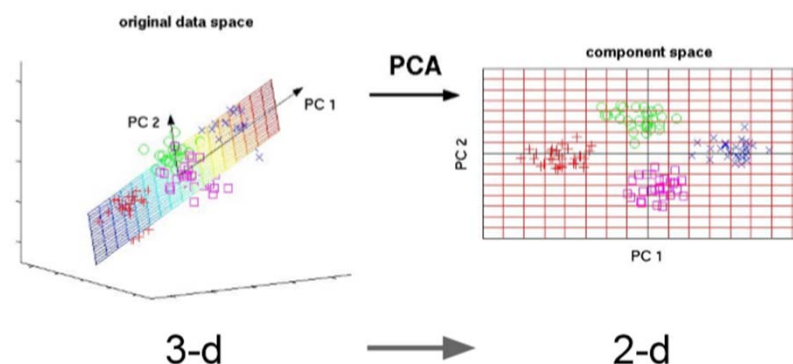
■ 回顾：有监督学习 vs 无监督学习

➤ 无监督学习 (Unsupervised Learning)

- 无标签数据: x
- 目标：学习数据中某些隐含或潜在的结构
- 例：聚类、降维、特征学习、密度估计等



K-means clustering



Principal Component Analysis
(Dimensionality reduction)

■ 回顾：有监督学习 vs 无监督学习

➤ 无监督学习 (Unsupervised Learning)

- 无标签数据: x 训练数据廉价易得
- 目标：学习数据中某些隐含或潜在的结构
- 例：聚类、降维、特征学习、密度估计等

解决无监督学习
=>理解视觉世界中的结构

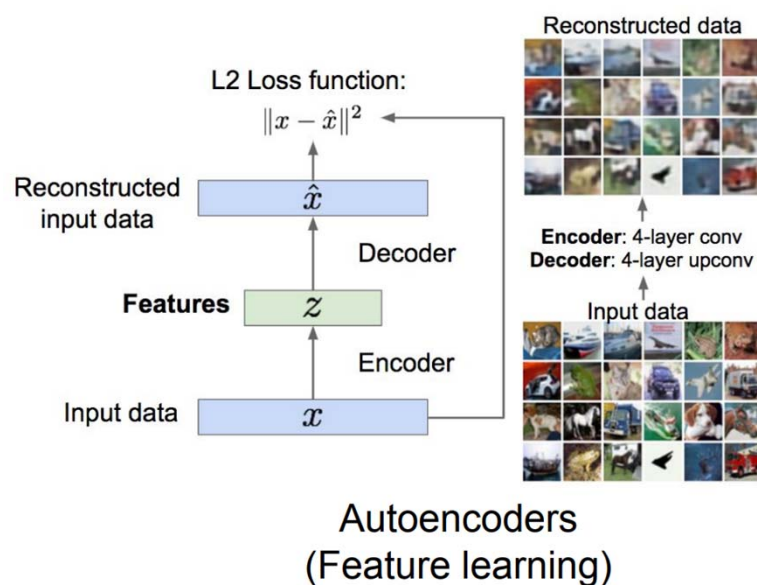
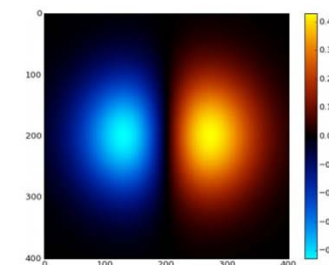
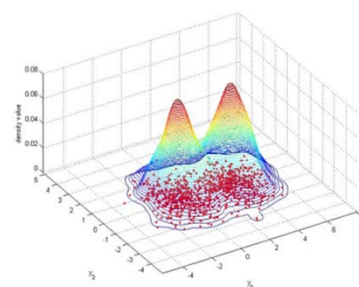


Figure copyright Ian Goodfellow, 2016. Reproduced with permission.

1-d density estimation



2-d density estimation

■ 生成模型 (Generative Model)

➤ 一系列用于随机生成可观察数据的模型

假设在一个连续或离散的高维空间 $\mathcal{X}$ 中, 存在一个随机向量 $\mathbf{X}$ 服从一个未知的数据分布 $p_r(\mathbf{x})$, $\mathbf{x} \in \mathcal{X}$. 生成模型是根据一些可观测的样本 $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}$ 来学习一个参数化的模型 $p_\theta(\mathbf{x})$ 来近似未知分布 $p_r(\mathbf{x})$, 并可以用这个模型来生成一些样本, 使得“生成”的样本和“真实”的样本尽可能地相似

- 训练阶段, 使模型学习训练数据的真实分布 p_{data}

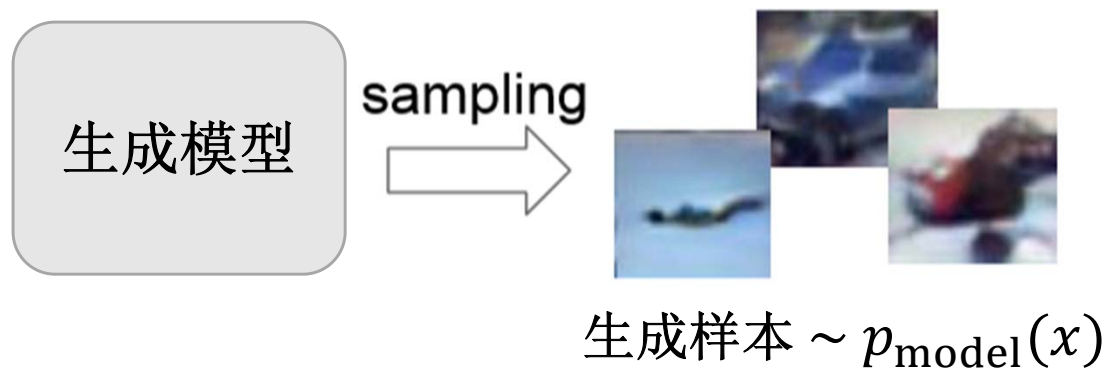


训练数据 $\sim p_{data}(x)$

■ 生成模型 (Generative Model)

➤ 一系列用于随机生成可观察数据的模型

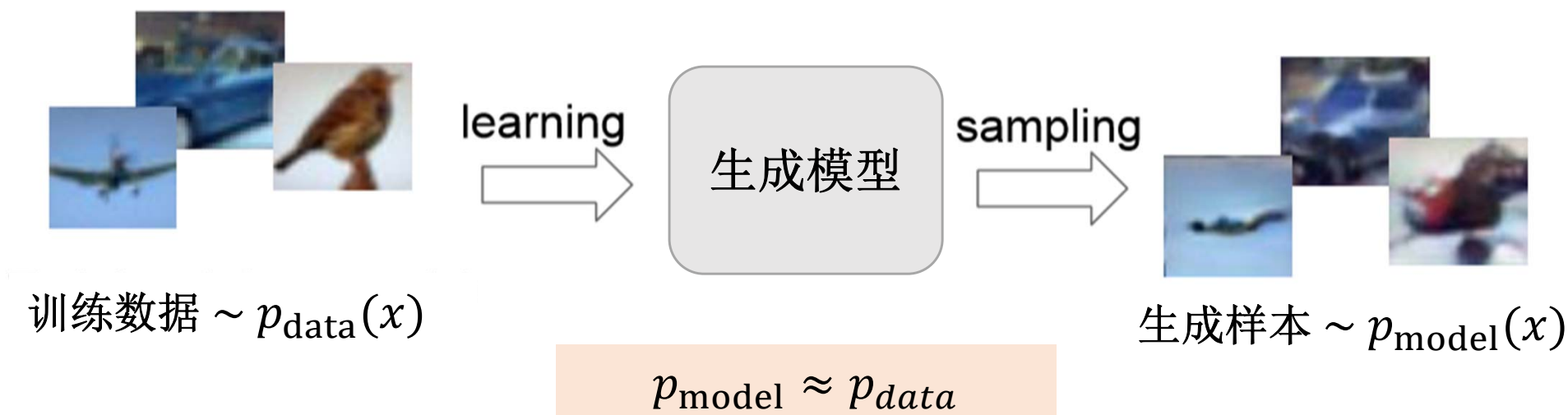
- 测试阶段，学好的模型能够随机生成新的数据



■ 生成模型 (Generative Model)

➤ 一系列用于随机生成可观察数据的模型

- 希望模型生成的新数据与训练数据的真实分布尽可能地相似



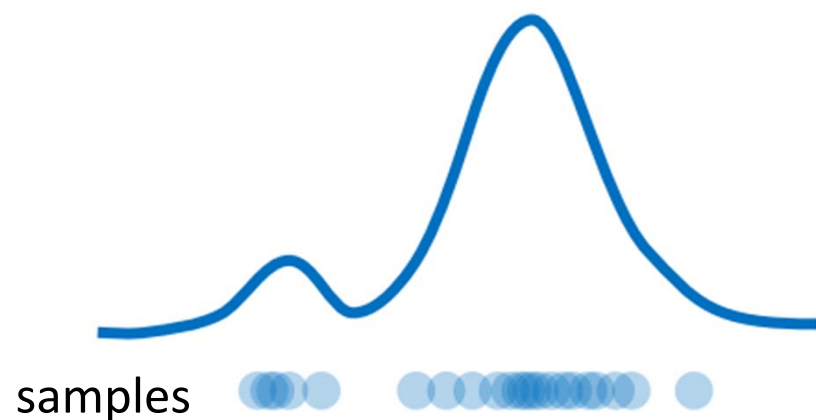
强调**密度估计**（无监督学习的一个核心问题），即数据分布的估计

■ 生成模型 (Generative Model)

➤ 密度估计 (density estimation)

- 显式密度估计：明确地定义模型，直接从样本集中估计概率分布
- 隐式密度估计：不需要直接估计概率分布，而是通过拟合模型 p_{model} ，使其能够生成符合数据分布 p_{data} 的样本

Density Estimation

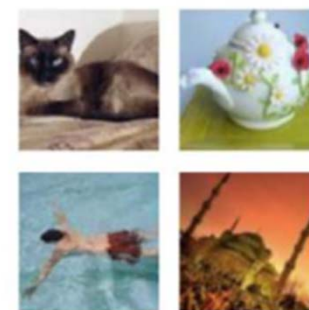


Sample Generation



Input samples

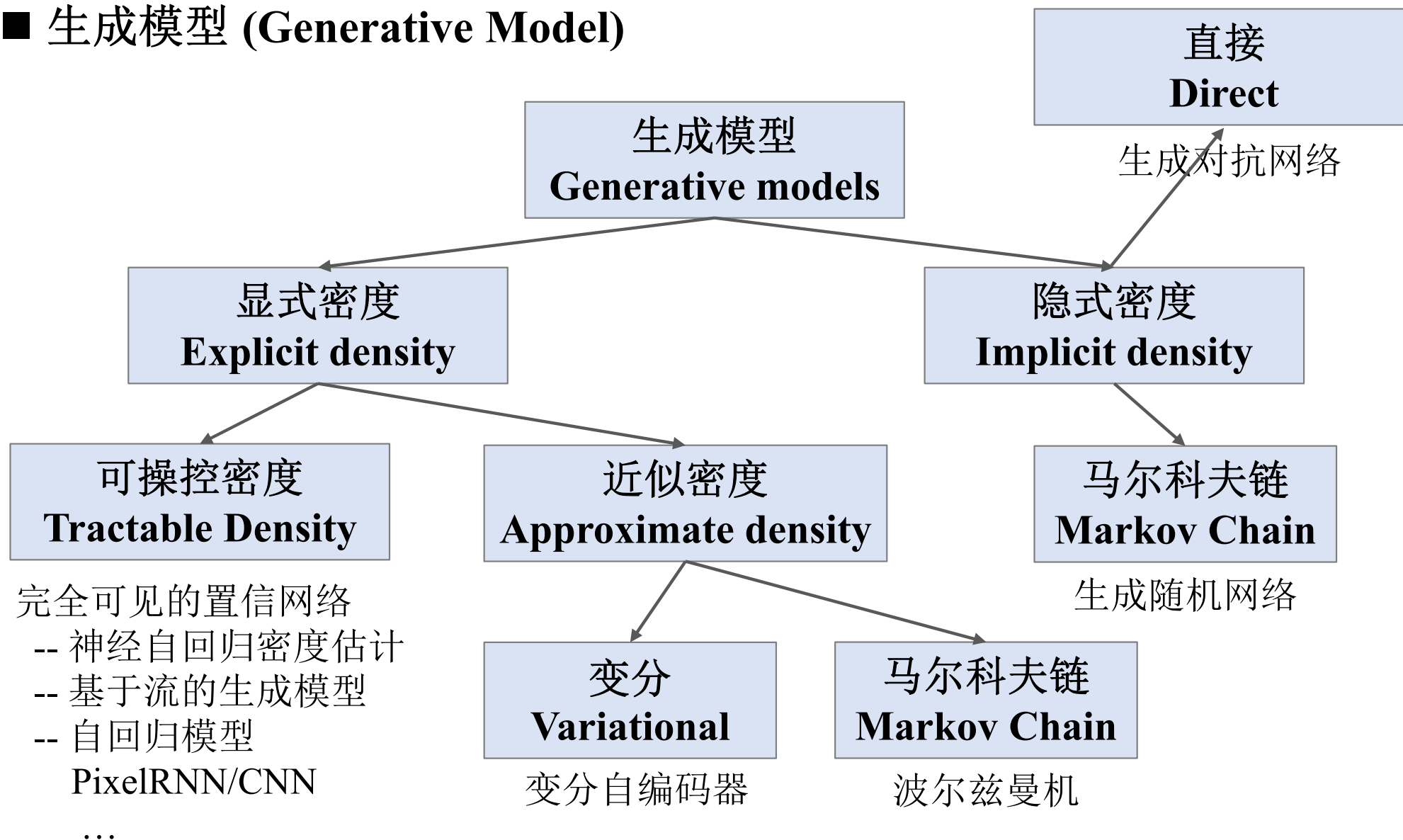
Training data $\sim P_{\text{data}}(x)$



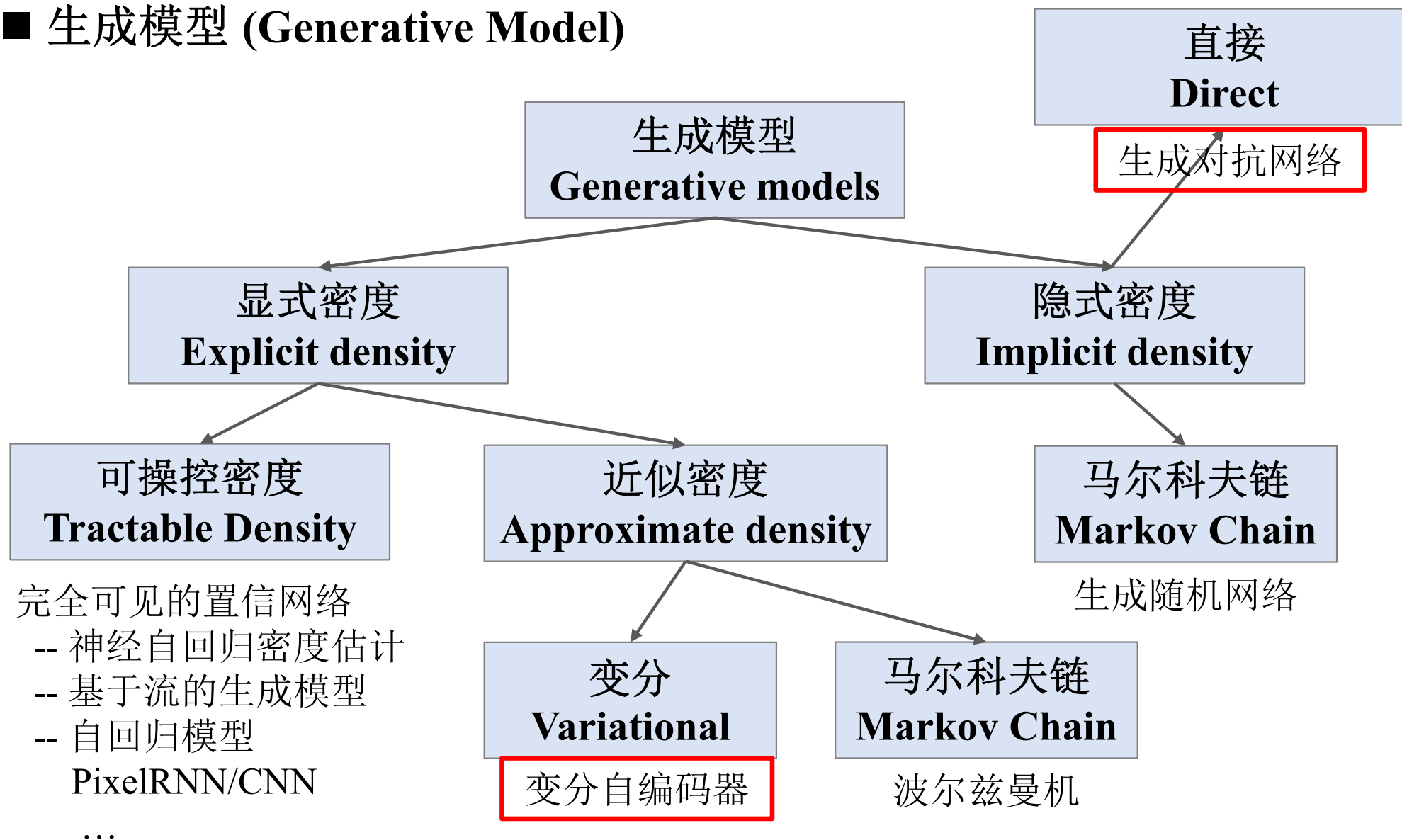
Generated samples

Generated $\sim P_{\text{model}}(x)$

■ 生成模型 (Generative Model)



■ 生成模型 (Generative Model)



■ 为何研究生成模型？

- 一些任务需要产生看起来真实的样本，如超分辨率重建、图像转换

低分辨率→高分辨率



input



output

Ledig *et al.*, Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network. CVPR2017.

素描→照片



input



output

Isola *et al.*, Image-to-Image Translation with Conditional Adversarial Networks. CVPR2017.

■ 为何研究生成模型？



- 更有效地表征和操控高维概率分布的数据 (如物理世界、医学影像等)
- 学习有效的隐含特征，为其他下游任务 (如分类) 提供服务
- 生成物理世界中的时序数据，用于机器人仿真和规划（强化学习）
- 利用有损数据（部分样本无标记）进行半监督学习，降低获得数据样本的难度
- 以及更多。。。

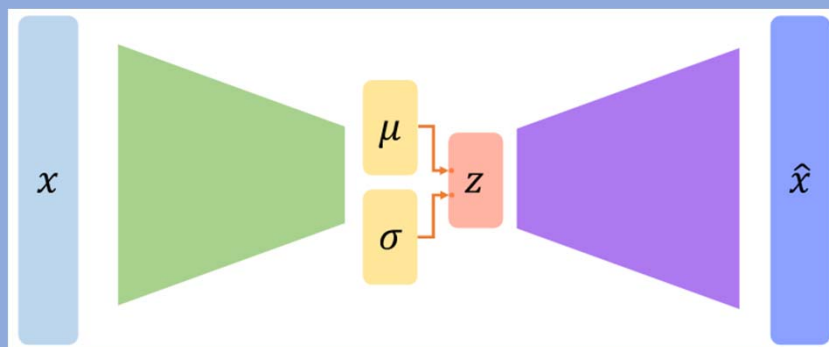


Richard Feynman

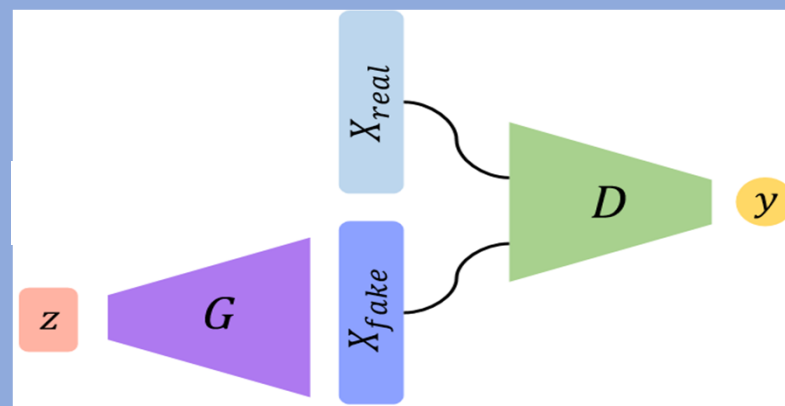
- 物理学家费曼：凡是我不能亲自创造出来的，我就不是真正理解
(What I cannot create, I do not understand)
- 生成模型：凡是我理解的东西，我就一定能够制造或者生成它
(What I understand, I can create)

■ 两种生成模型

变分自编码器 (Variational Autoencoder, AVE)



生成对抗网络 (Generative Adversarial Network, GAN)



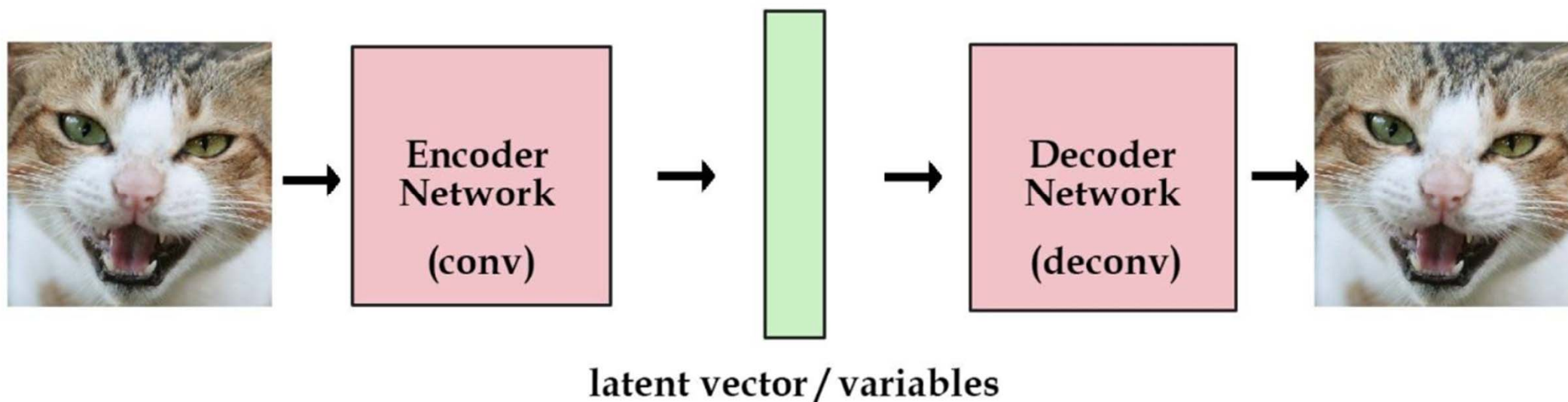
PART TWO

变分自编码器

Variational AutoEncoder

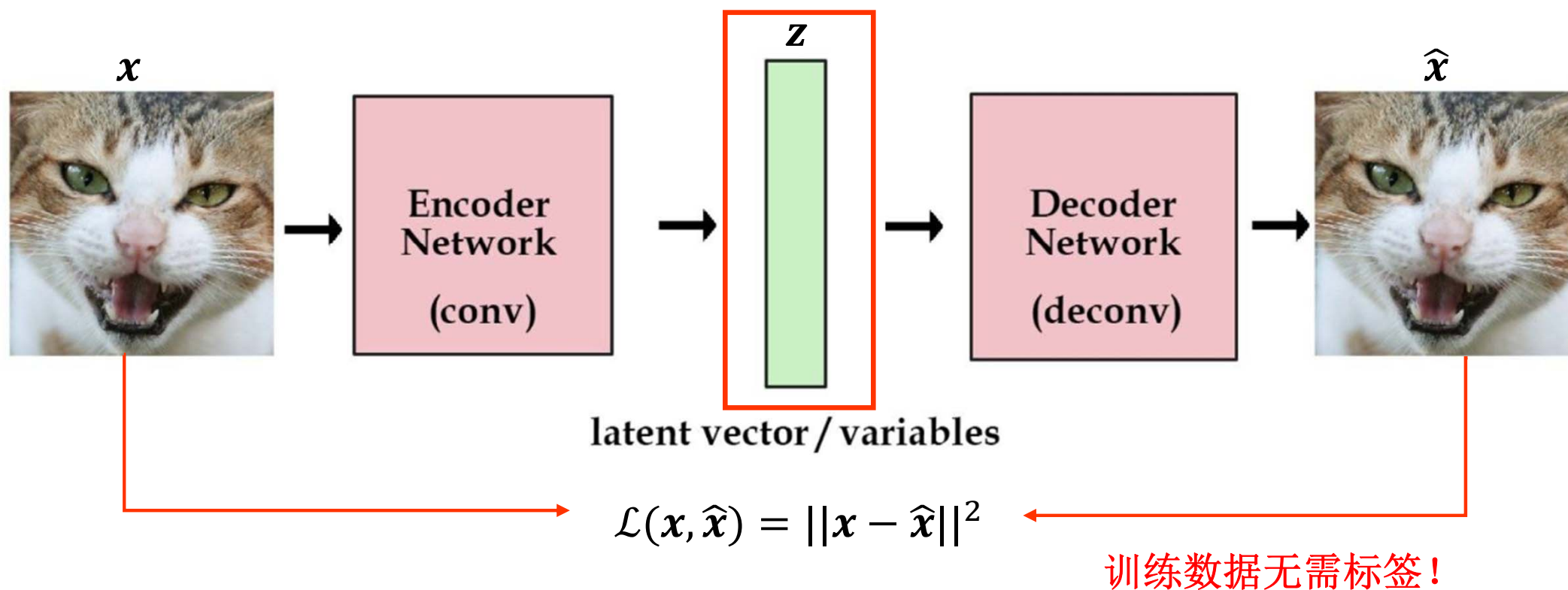
■ 回顾：自编码器 (Autoencoder)

- 编码器 (Encoder) 将输入数据映射为一个隐含变量
- 解码器 (Decoder) 从隐含变量中重构出原始输入数据

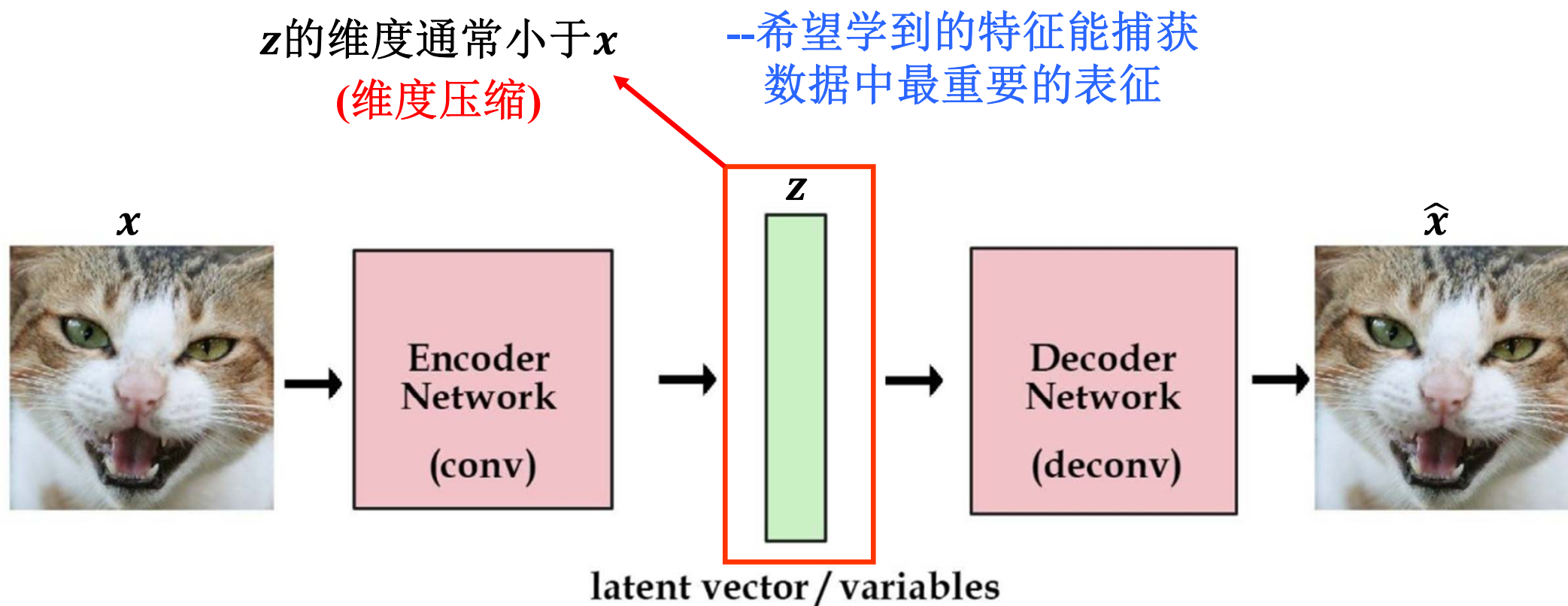


■ 回顾：自编码器 (Autoencoder)

- 最小化重构误差，使网络学会捕获对数据的隐含变量



■ 回顾：自编码器 (Autoencoder)

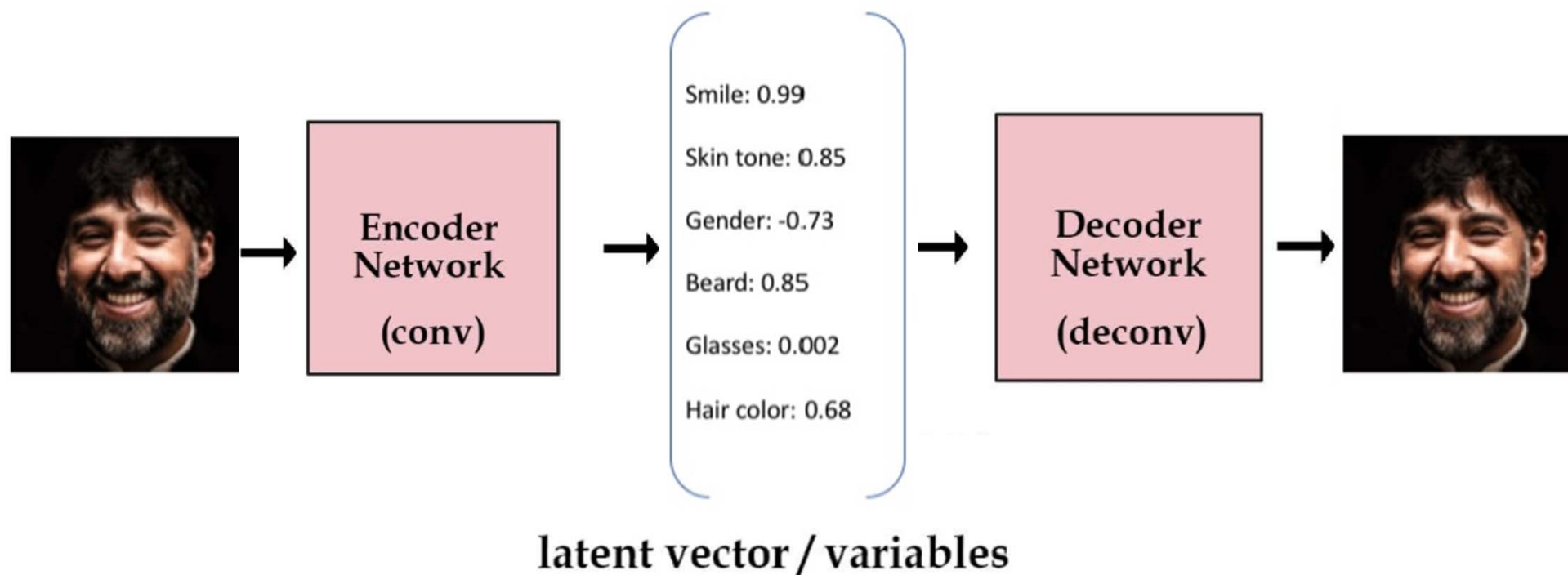


自编码是一种数据压缩方式

Autoencoding = **Automatically encoding** data

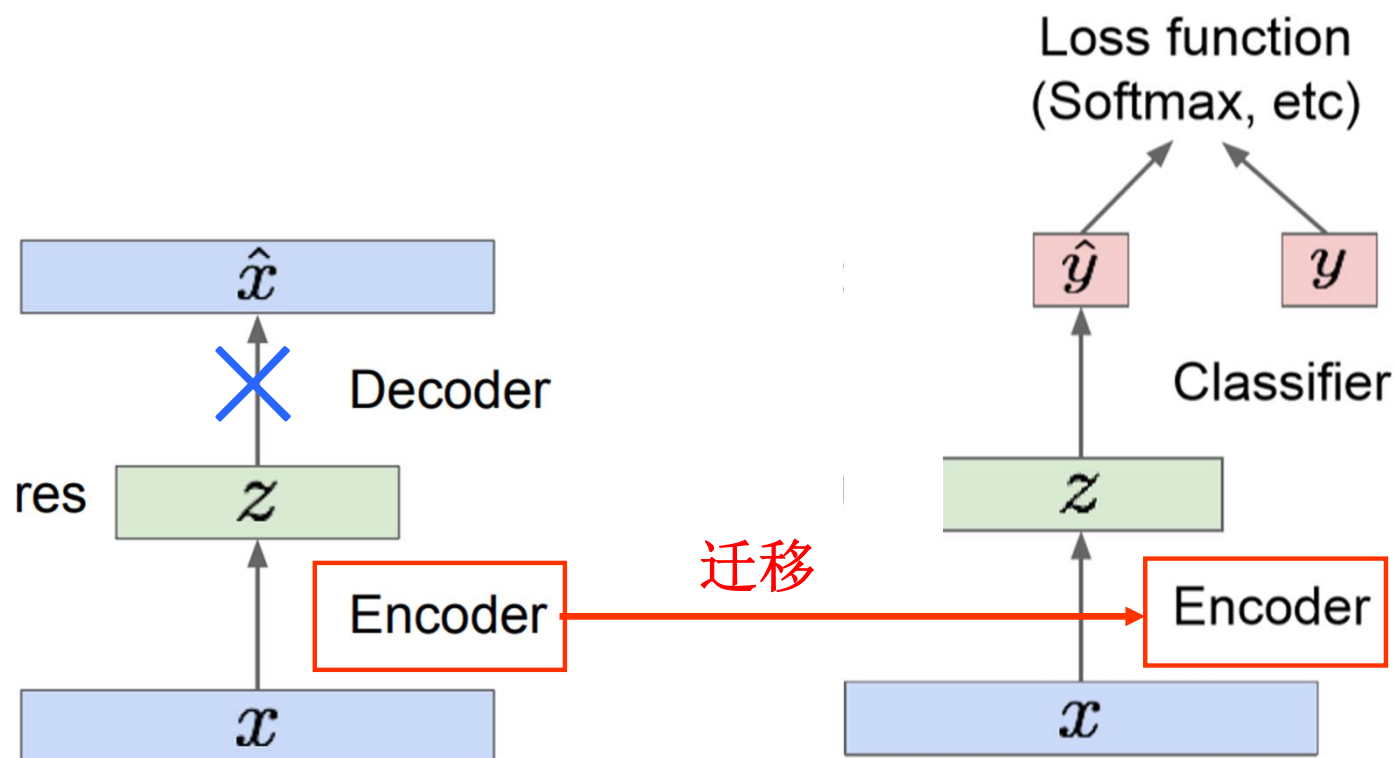
■ 回顾：自编码器 (Autoencoder)

- 隐含变量可表征输入数据的某些结构或属性

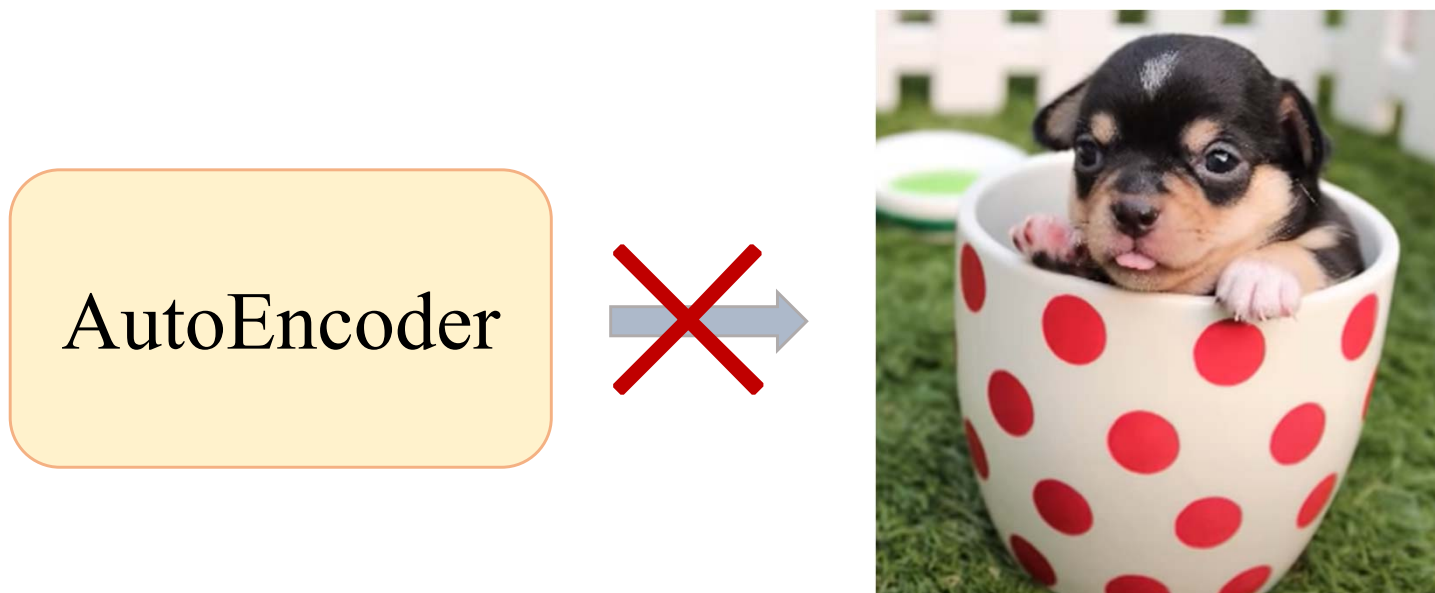


■ 回顾：自编码器 (Autoencoder)

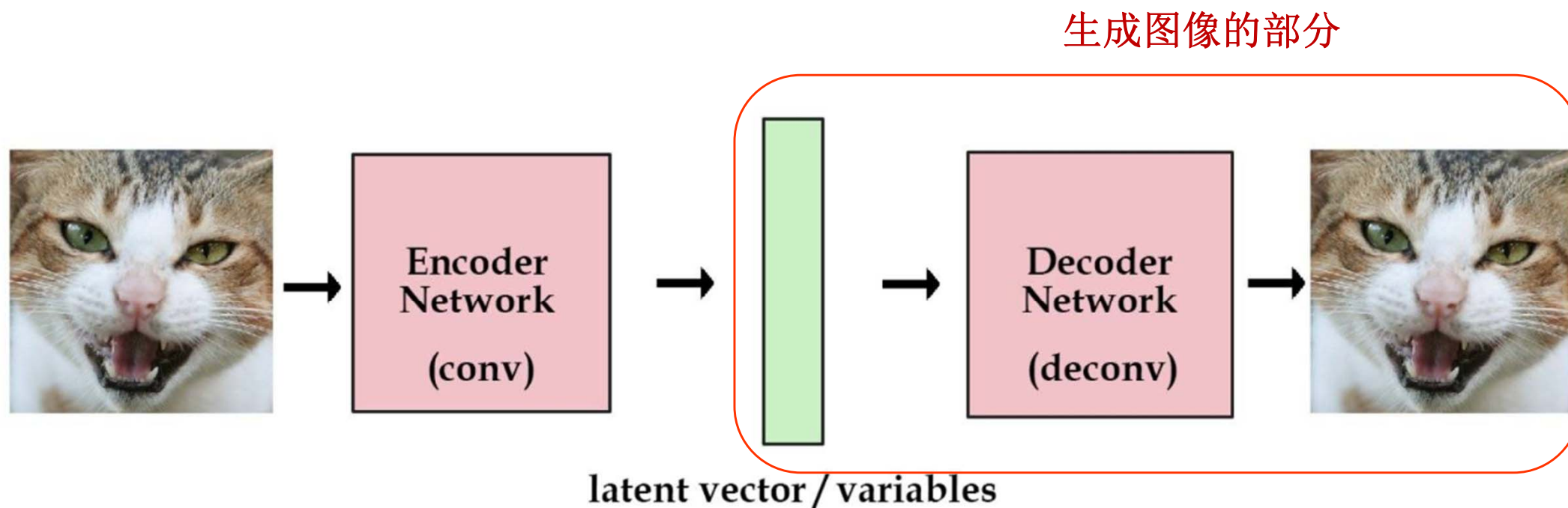
- 训练完成后，Encoder能够将训练数据映射至一个有效地隐含空间
- 可抛开Decoder，将学好的Encoder迁移至其他任务中



■ 自编码器是否能生成新的图像？



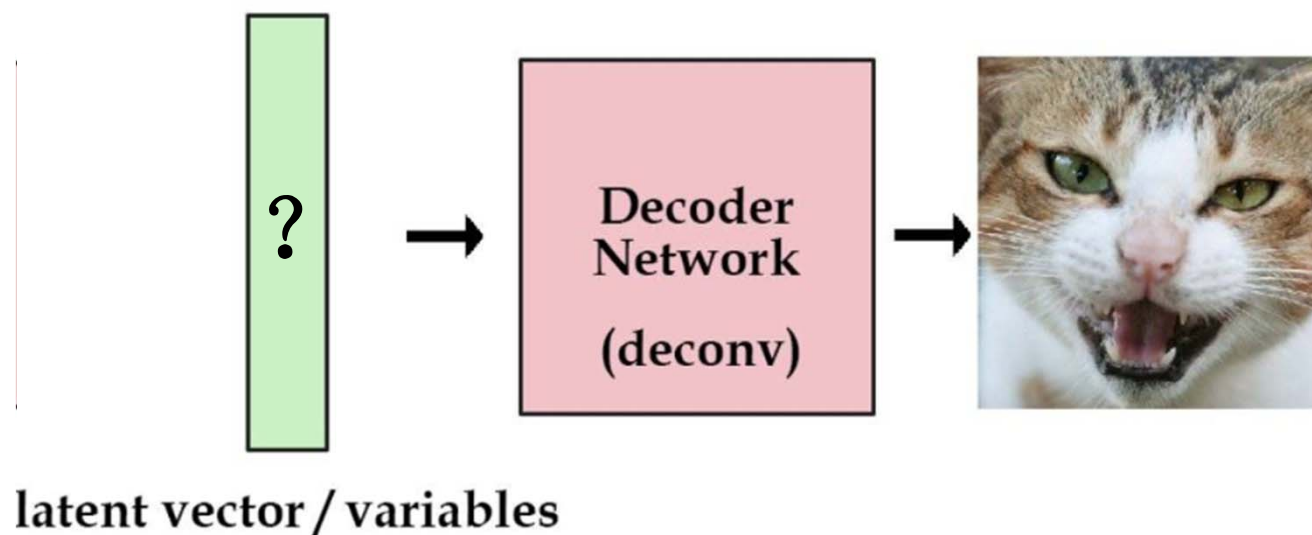
■ 自编码器是否能生成新的图像？



■ 自编码器是否能生成新的图像？

➤ 测试阶段

- 不知道如何创建需要输入的隐变量

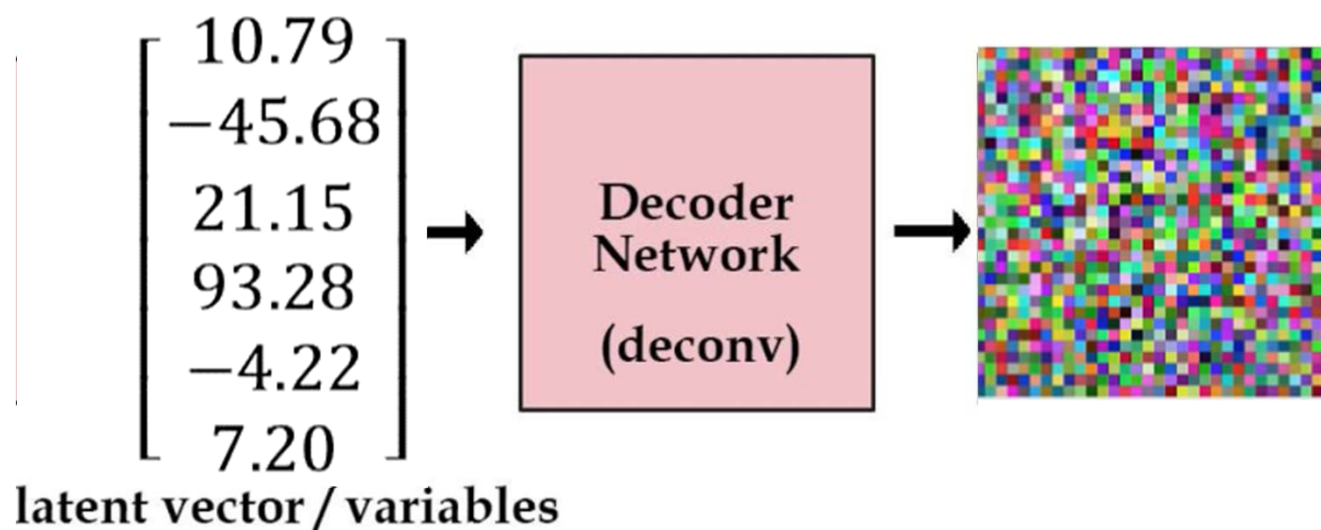


生成图像的部分

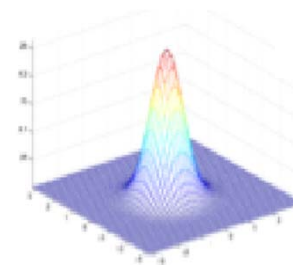
■ 自编码器是否能生成新的图像？

➤ 测试阶段

- 假设输入一组随机的向量，我们很大程度上将得到一个噪声图像



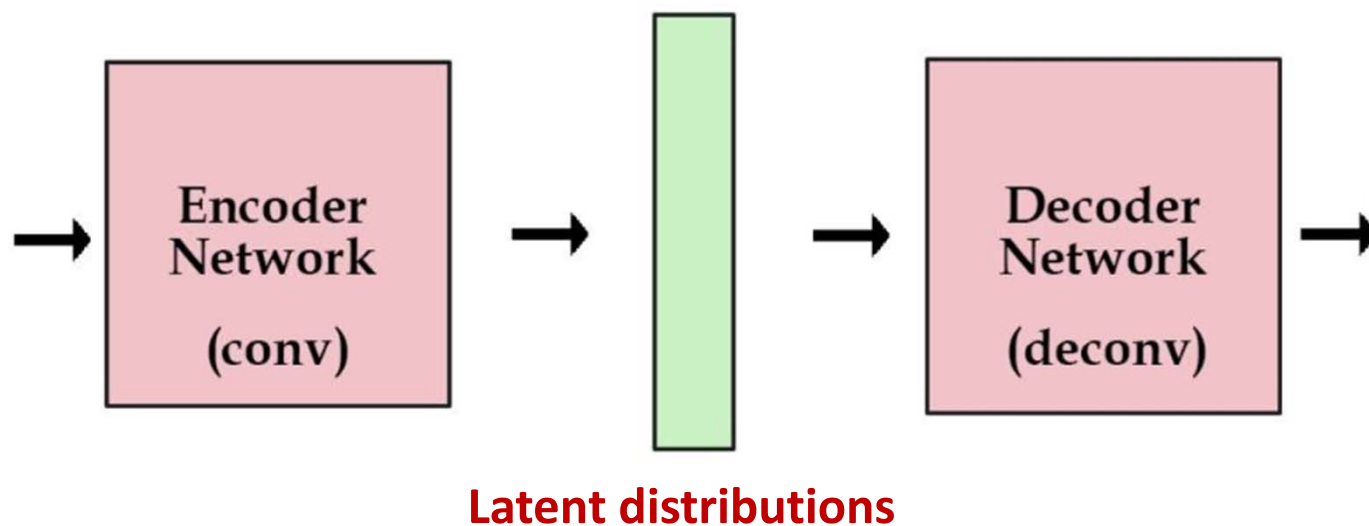
生成图像的部分

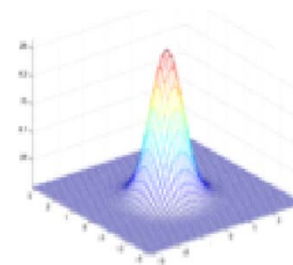


■ 自编码器是否能生成新的图像？

➤ 解决方案

- 对Encoder增加约束，强制其产生的隐含变量基本服从某种分布

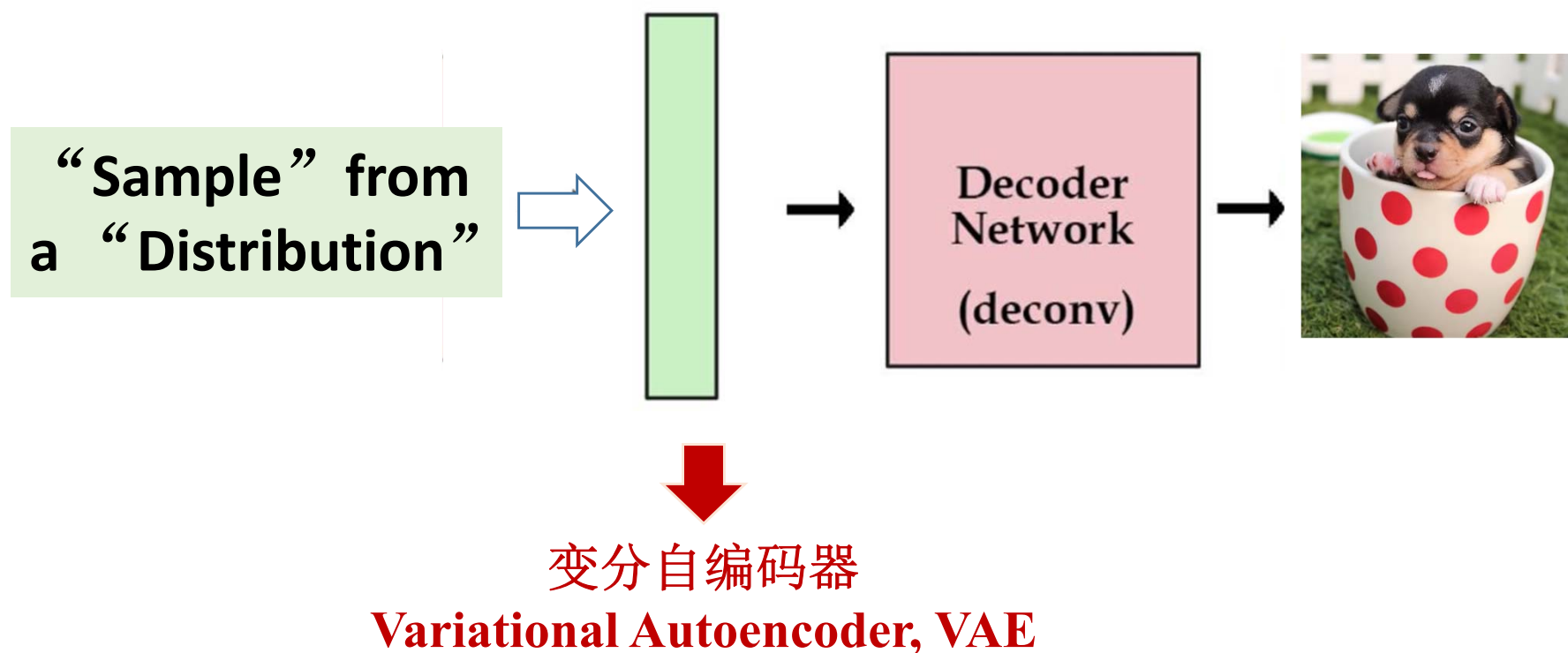




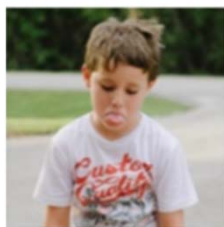
■ 自编码器是否能生成新的图像？

➤ 解决方案

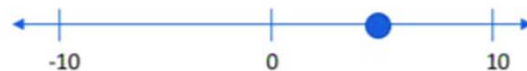
- 测试时，从该分布中采样作为隐含量，并传递到解码器，生成新图像



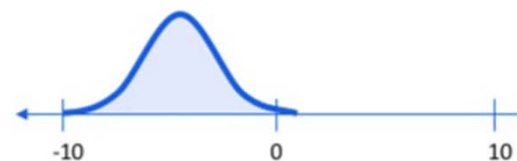
- 隐含变量的每个元素由原先的“单值” $\Rightarrow$ “取值的概率分布”



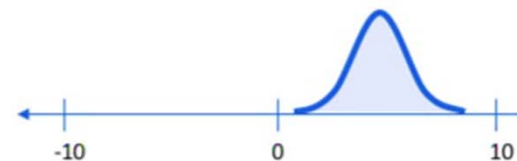
微笑（离散数值）



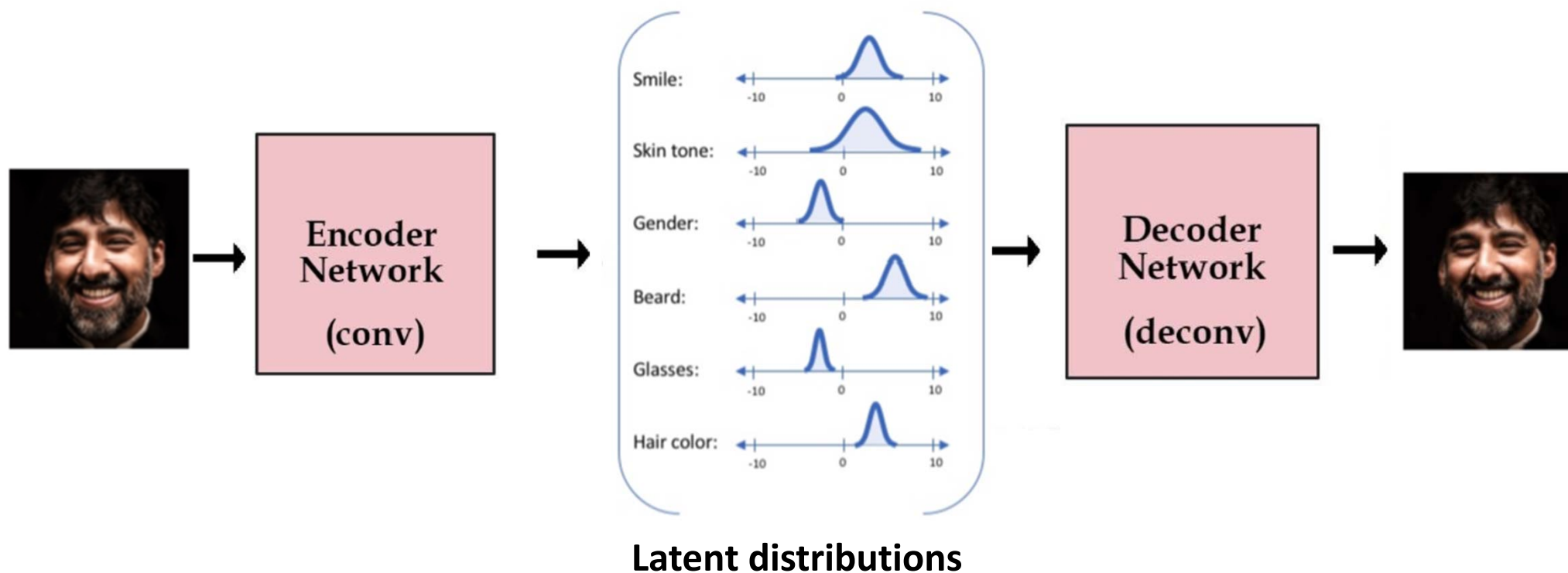
微笑（概率分布）



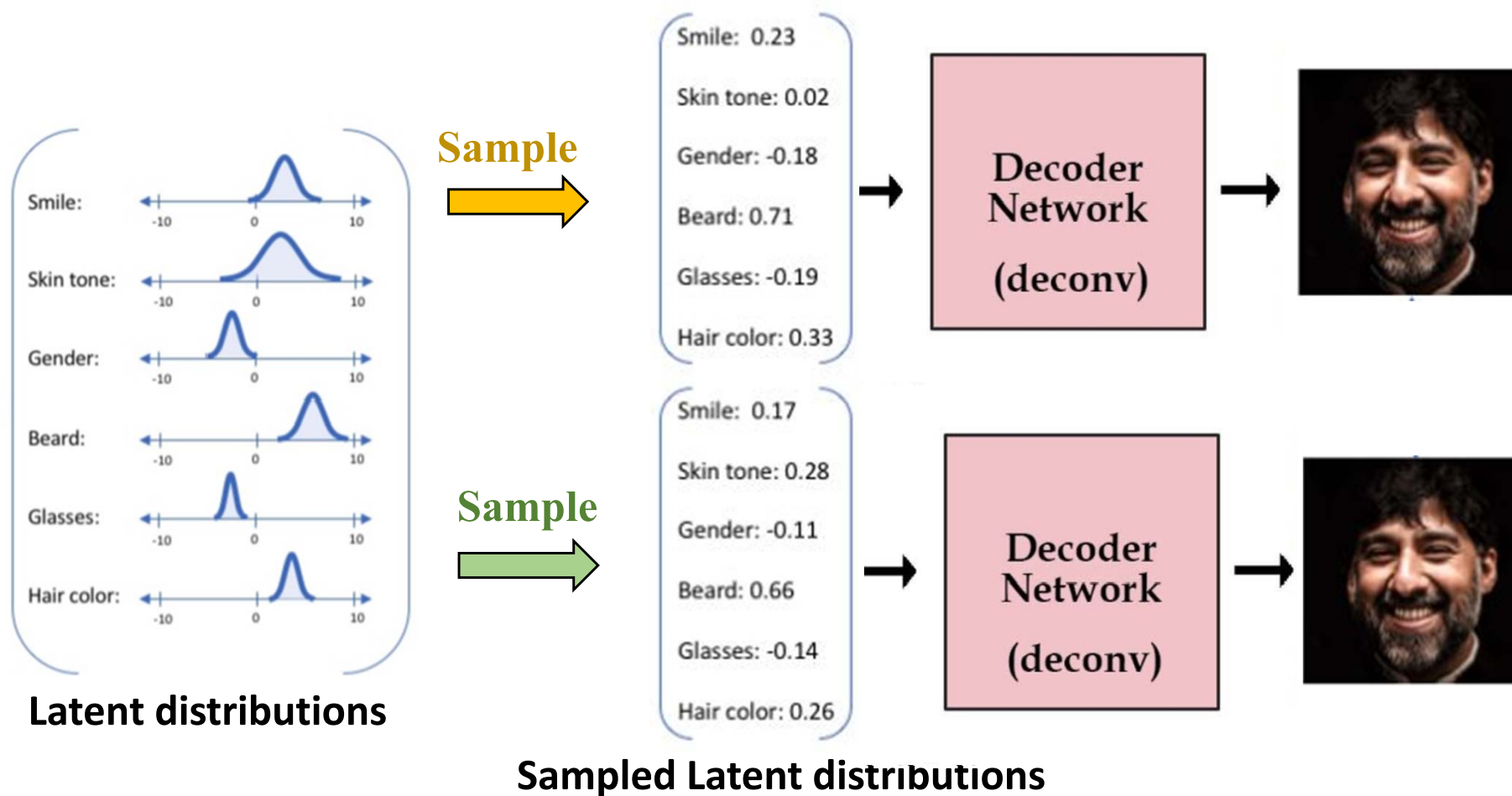
vs.



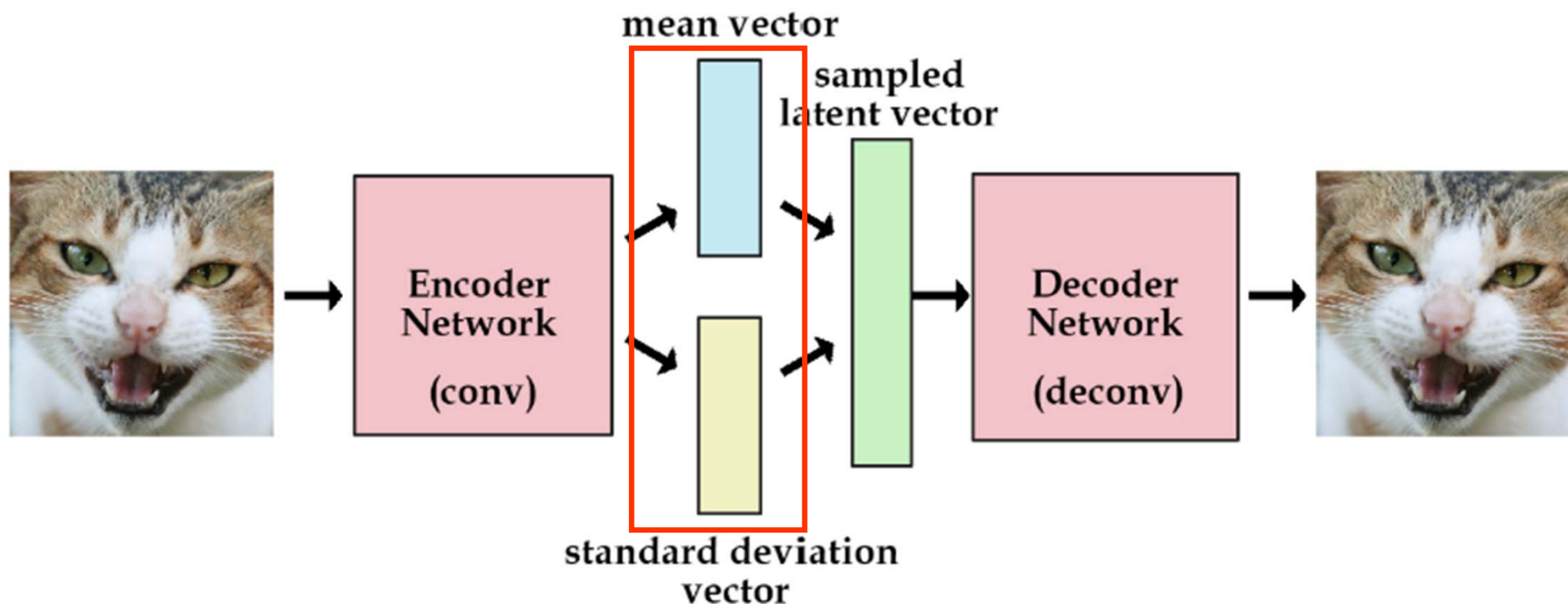
- 生成图像（即对隐含变量解码）时，从每个隐含分布中随机采样，生成一个向量作为解码器模型的输入



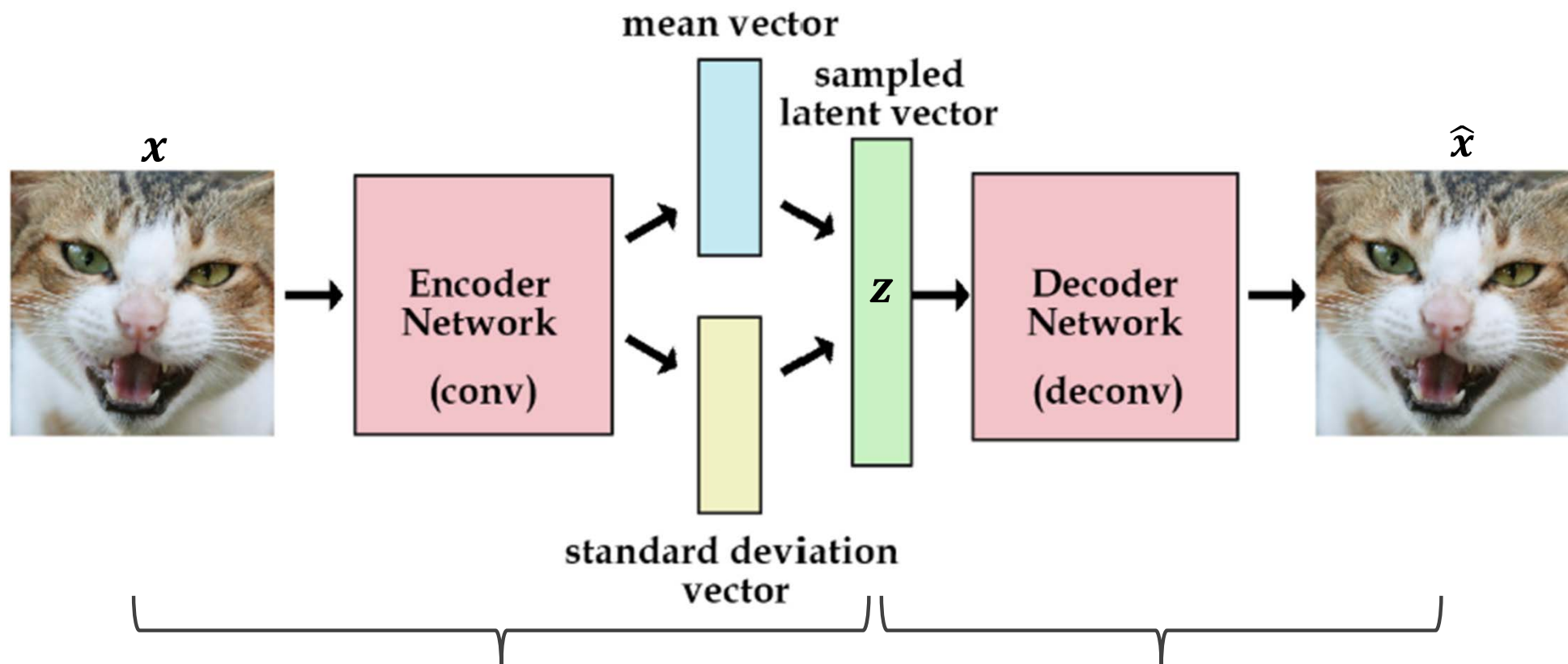
- 在连续平滑的隐含分布空间随机采样，彼此相邻的值具有类似重构



- 通常假设采样的隐含变量服从正态分布 $\mathcal{N}(\mu, \sigma^2)$
- 从均值分布和标准差分布中随机抽取，计算隐含变量



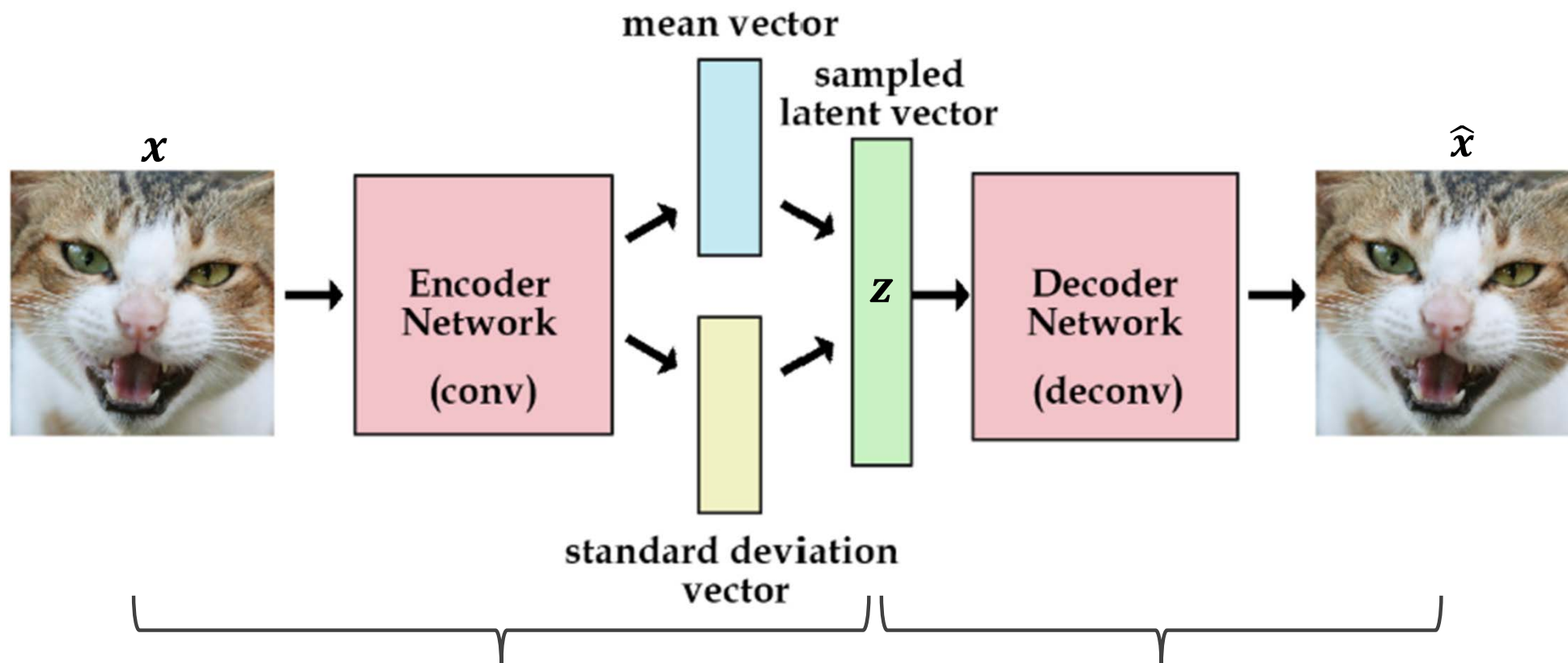
- VAE是一种概率形式的自编码器



z 的后验分布的推断过程, 即 $p_{\phi}(z|x)$

$\hat{x}$ 的条件分布生成过程, 即 $q_{\theta}(\hat{x}|z)$

■ 模型优化

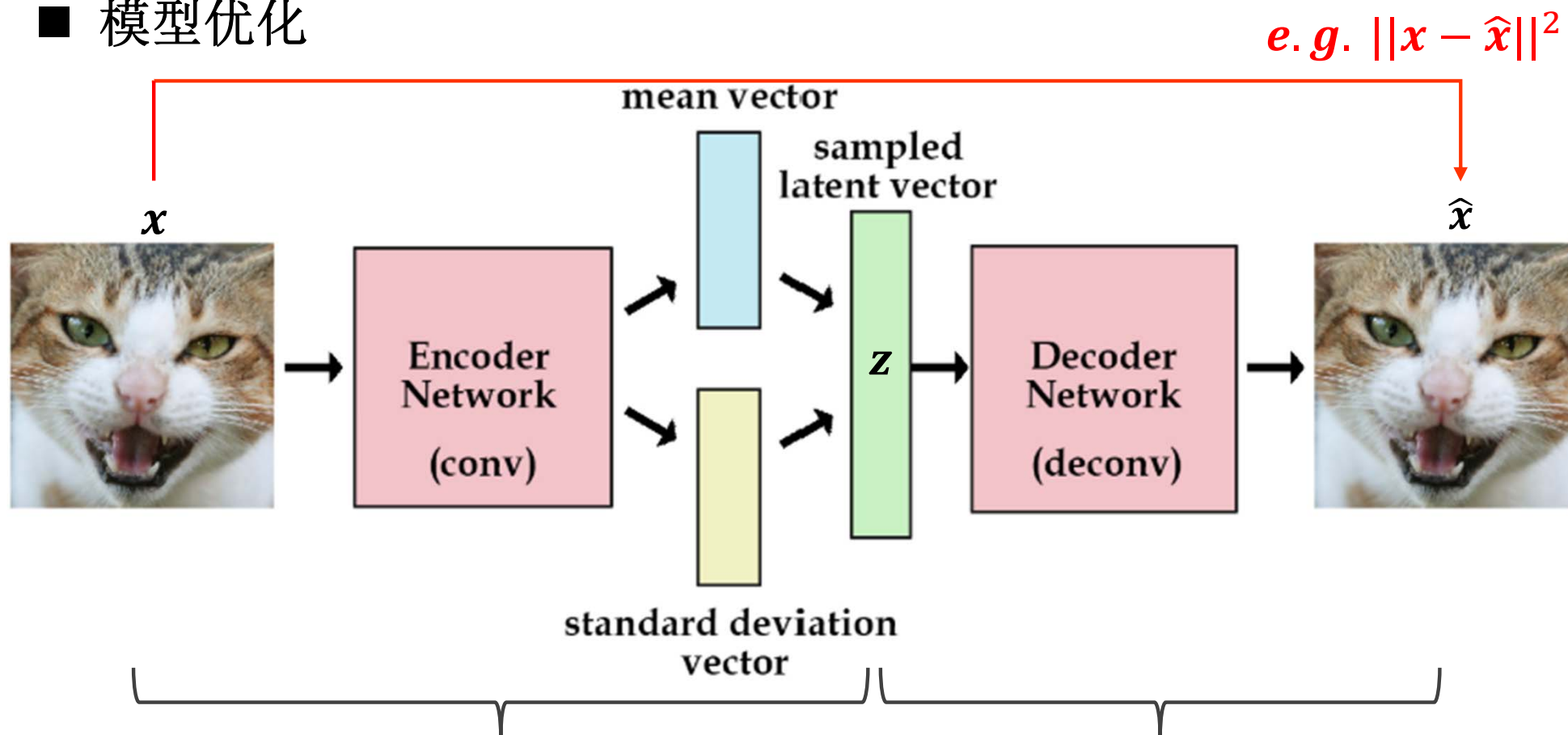


z 的后验分布的推断过程, 即 $p_{\phi}(z|x)$

$\hat{x}$ 的条件分布生成过程, 即 $q_{\theta}(\hat{x}|z)$

$$\mathcal{L}(\phi, \theta) = \text{重构误差} + \text{约束项}$$

■ 模型优化



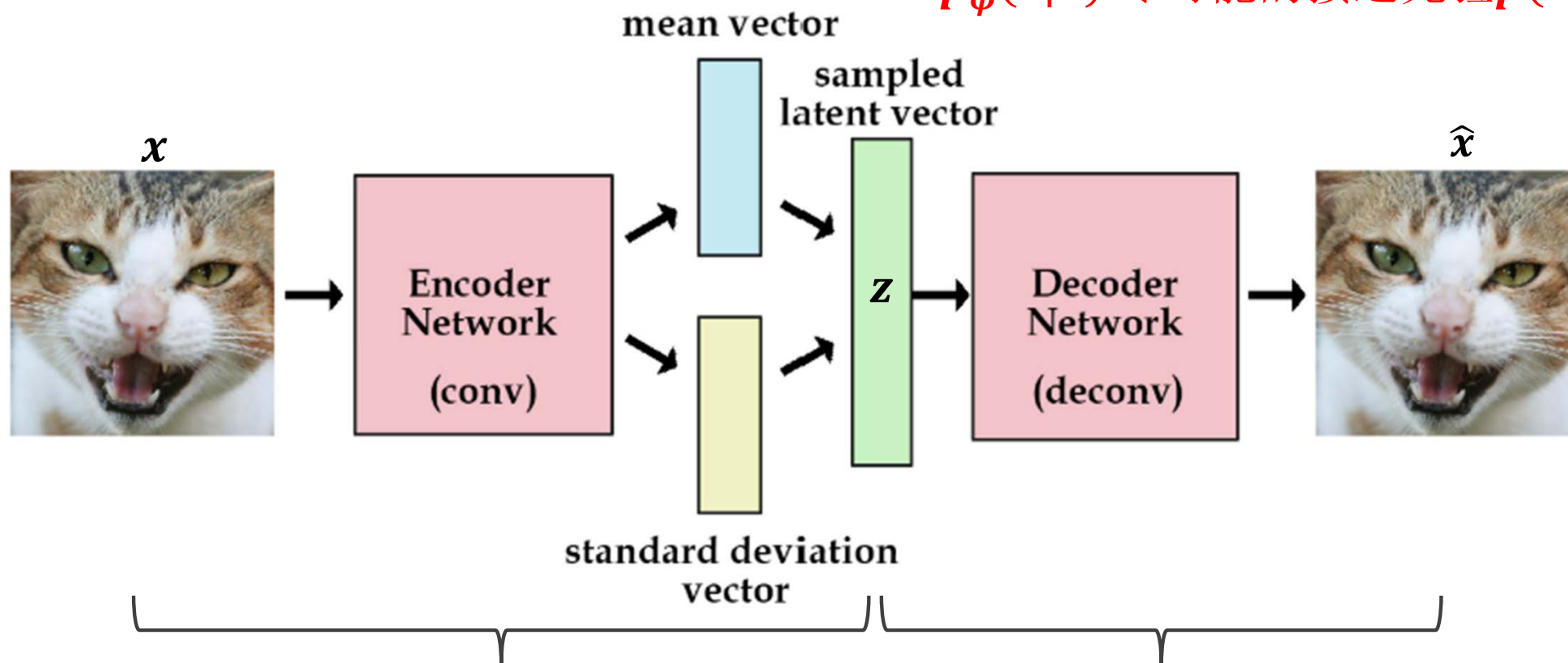
z 的后验分布的推断过程, 即 $p_{\phi}(z|x)$

$\hat{x}$ 的条件分布生成过程, 即 $q_{\theta}(\hat{x}|z)$

$$\mathcal{L}(\phi, \theta) = \text{重构误差} + \text{约束项}$$

■ 模型优化

$p_{\phi}(z|x)$ 尽可能的接近先验 $p(z)$



z 的后验分布的推断过程, 即 $p_{\phi}(z|x)$

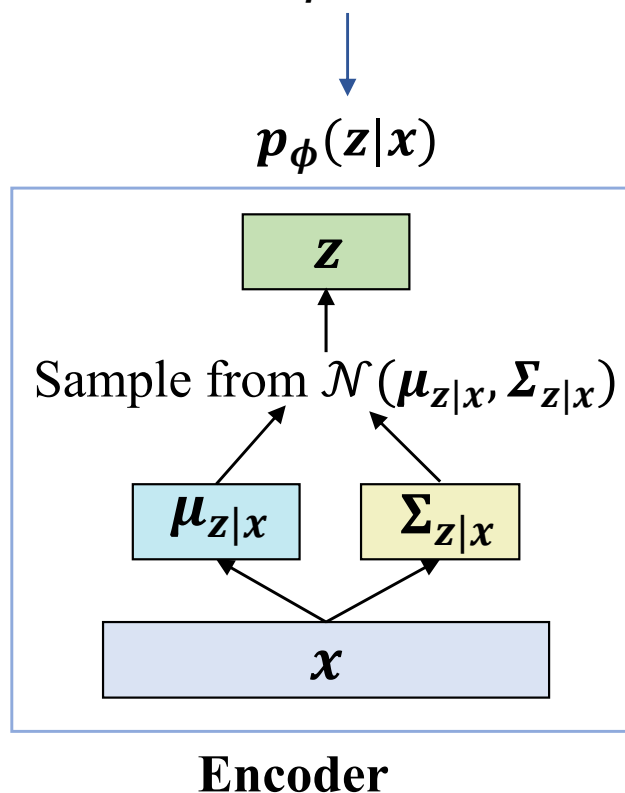
$\hat{x}$ 的条件分布生成过程, 即 $q_{\theta}(\hat{x}|z)$

$$\mathcal{L}(\phi, \theta) = \text{重构误差} + \text{约束项}$$

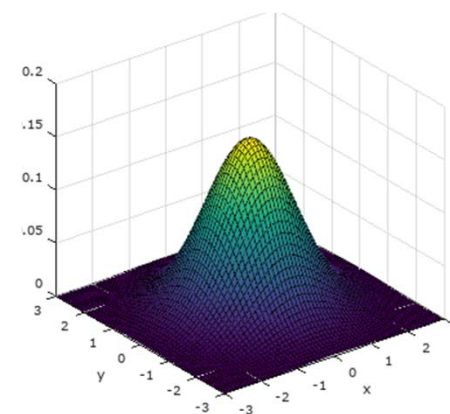
■ 模型优化

➤ 约束项

- 目标: $p_\phi(z|x)$ 尽可能的接近先验 $p(z)$



通常假设 $p(z) \sim \mathcal{N}(0,1)$



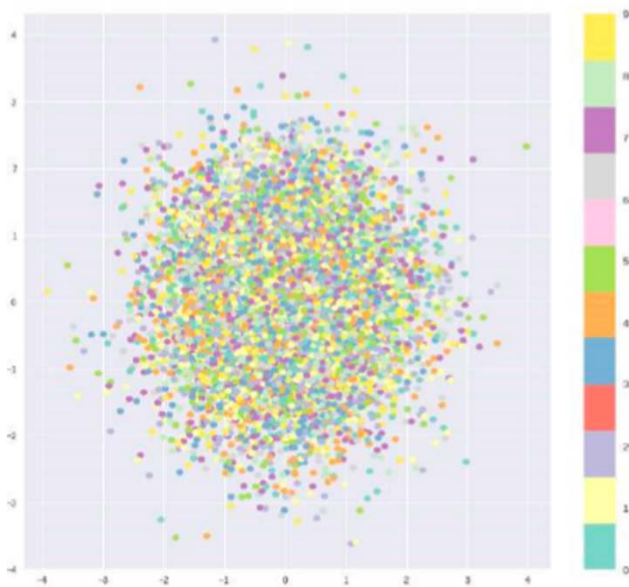
■ 模型优化

➤ 正则项

- 目标: $p_{\phi}(z|x)$ 尽可能的接近先验 $p(z)$

$$p(z) \sim \mathcal{N}(0,1)$$

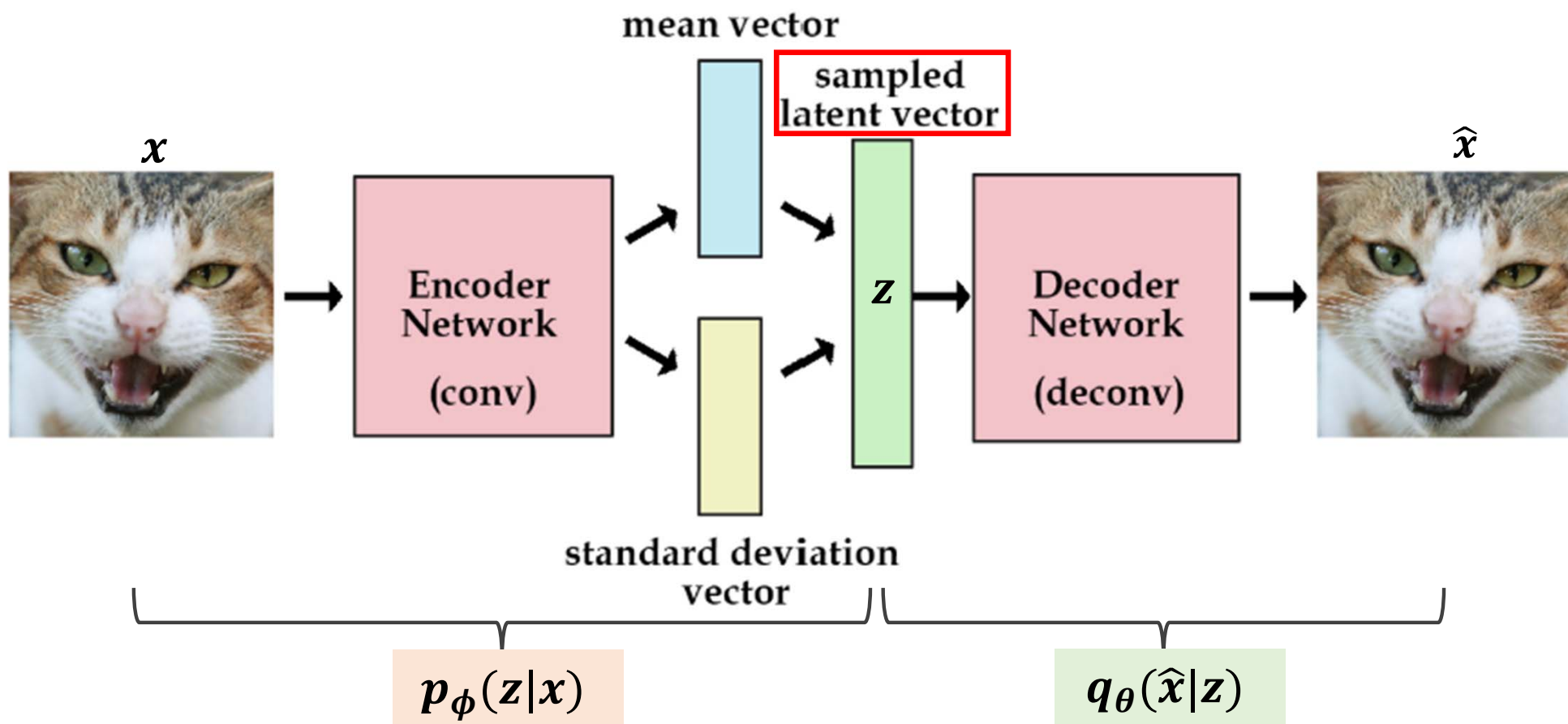
- 采用KL散度, 即:
$$\begin{aligned} D_{KL}(p_{\phi}(z|x) \parallel p(z)) \\ &= D_{KL}(\mathcal{N}(\mu_{z|x}, \Sigma_{z|x}) \parallel \mathcal{N}(0,1)) \\ &= -\frac{1}{2} \sum_j^K \Sigma_{z|x_j} + \mu_{z|x_j}^2 - 1 - \log \Sigma_{z|x_j} \end{aligned}$$



- 鼓励Encoder的输出分布 $p_{\phi}(z|x)$ 均衡地分布在隐变量的先验空间

■ 模型优化

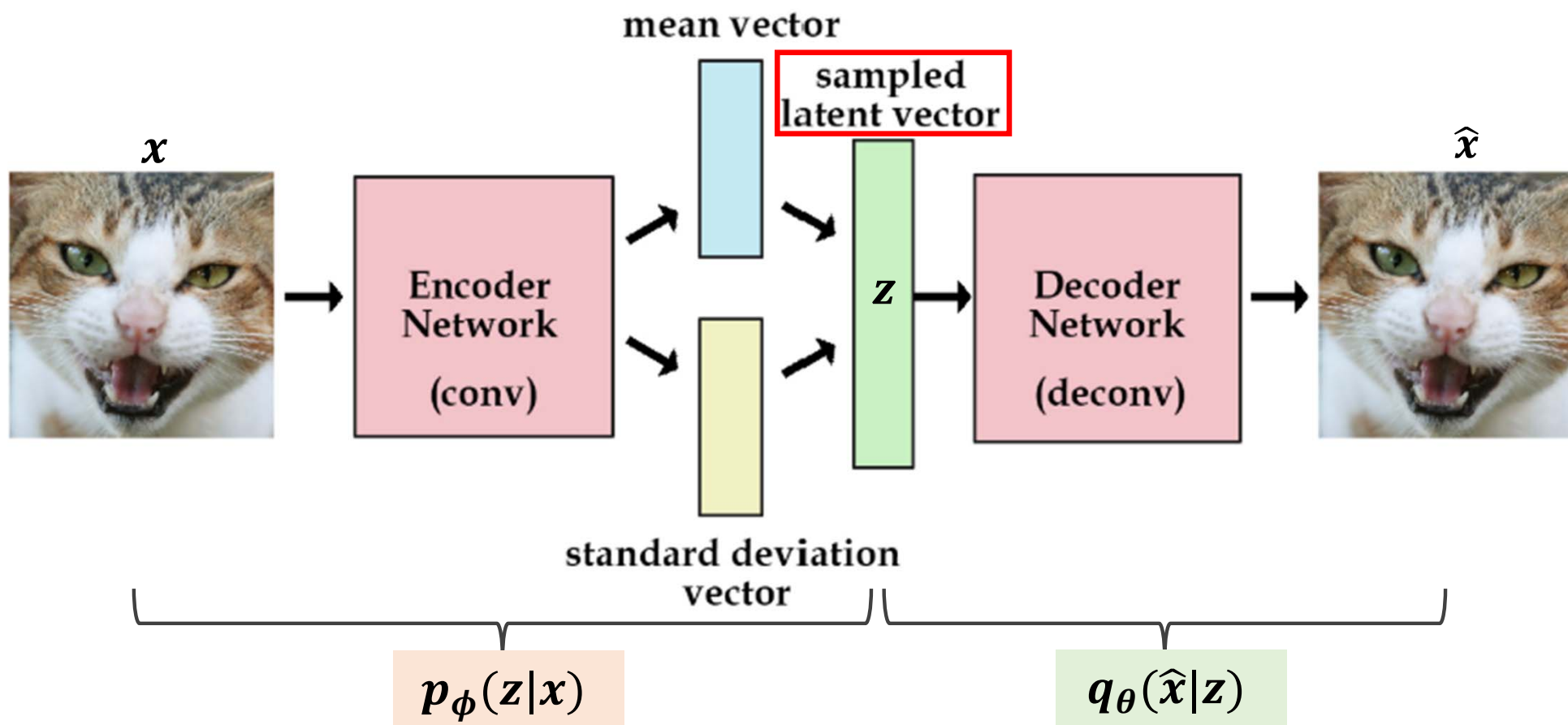
问题: **sampling layer**无法后向传递梯度!



$$\mathcal{L}(\phi, \theta) = \text{重构误差} + \text{正则项}$$

■ 模型优化

问题: **sampling layer**无法后向传递梯度!

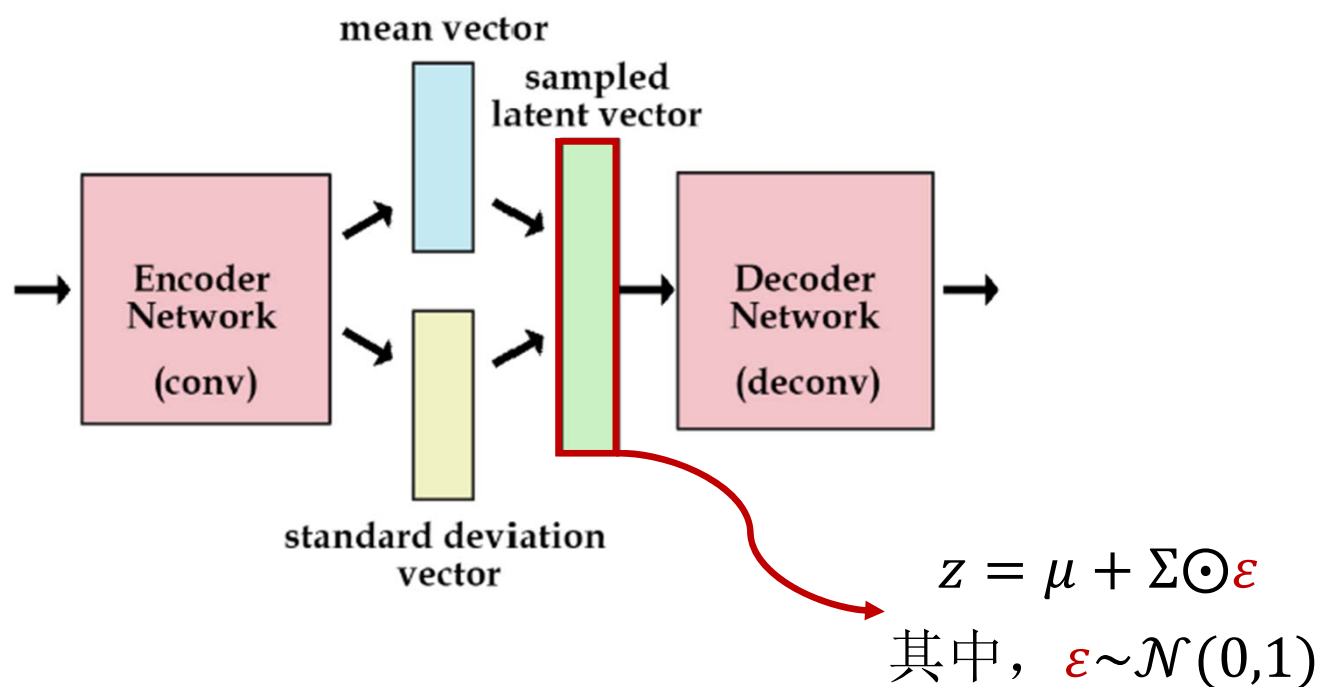


$$\mathcal{L}(\phi, \theta) = \text{重构误差} + \text{正则项}$$

■ 模型优化

➤ 问题: **Sampling layer**无法后向传递梯度!

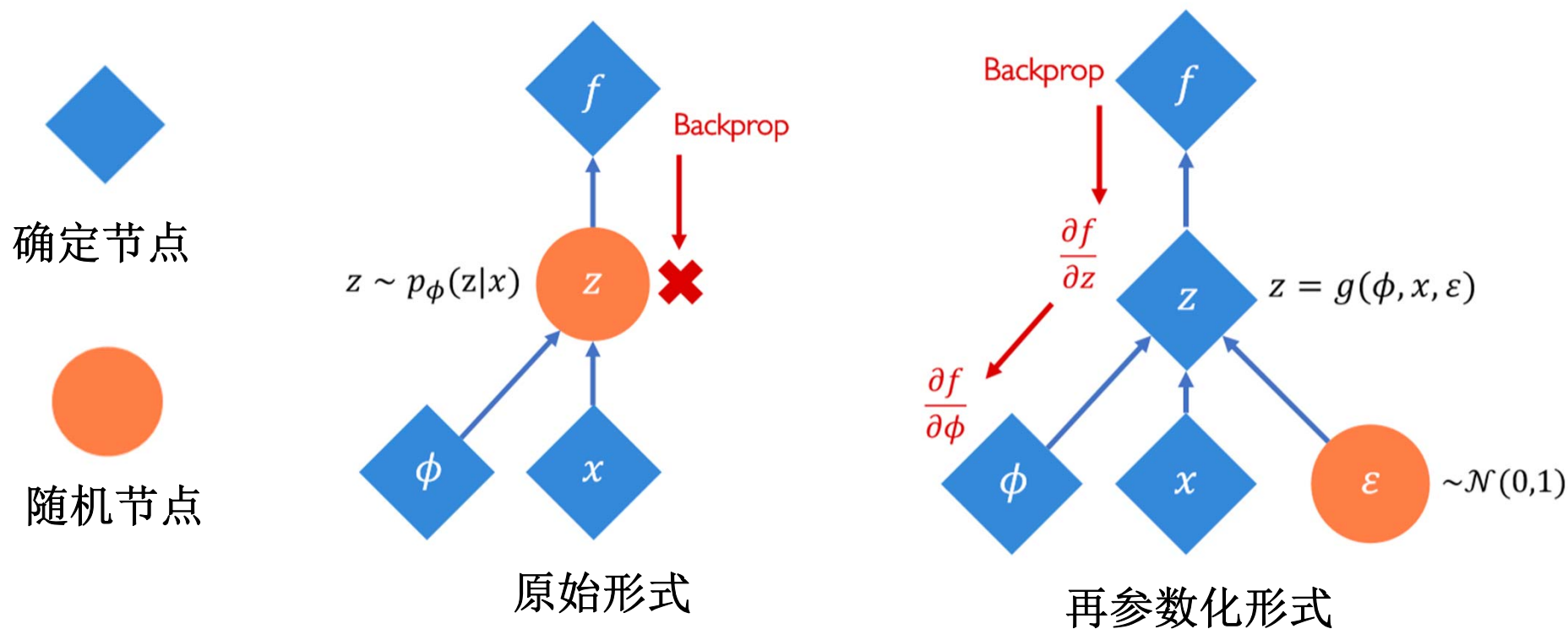
解决方案: 再参数化 (**Reparameterization Trick**)



■ 模型优化

➤ 问题: Sampling layer无法后向传递梯度!

解决方案: 对Sampling layer再参数化 (Reparameterization Trick)



■ 模型优化

```
import tensorflow as tf

def gaussian_MLP_encoder(x, n_hidden, n_output, keep_prob):

    ...

    gaussian_params = tf.matmul(h1, wo) + bo
    # The mean parameter is unconstrained
    mean = gaussian_params[:, :n_output]
    # The standard deviation must be positive:
    stddev = 1e-6 + tf.nn.softplus(gaussian_params[:, n_output:])

    return mean, stddev

def autoencoder(x_hat, x, dim_img, dim_z, n_hidden, keep_prob):

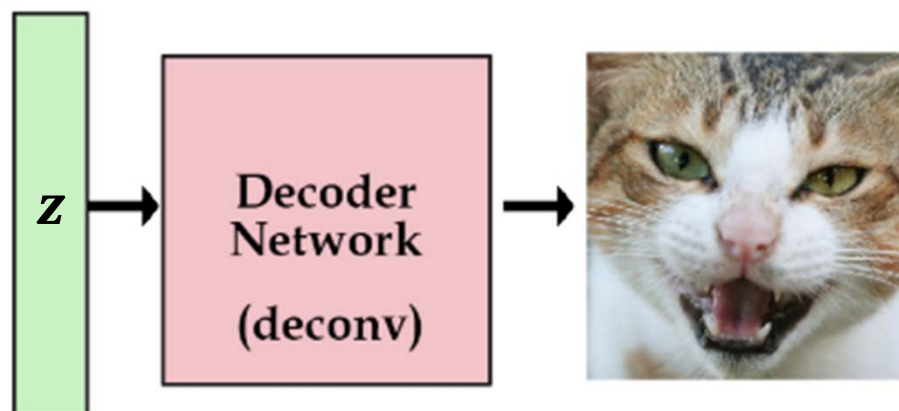
    # encoding
    mu, sigma = gaussian_MLP_encoder(x_hat, n_hidden, dim_z, keep_prob)
    # sampling by re-parameterization technique
    z = mu + sigma * tf.random_normal(tf.shape(mu), 0, 1, dtype=tf.float32)
    # decoding
    y = bernoulli_MLP_decoder(z, n_hidden, dim_img, keep_prob)

    # loss
    marginal_likelihood = tf.reduce_sum(x * tf.log(y) + (1 - x) * tf.log(1 - y), 1)
    KL_divergence = 0.5 * tf.reduce_sum(tf.square(mu) + tf.square(sigma) - tf.log(1e-8 + tf.square(sigma)) - 1, 1)

    ELBO = tf.reduce_mean(marginal_likelihood) - tf.reduce_mean(KL_divergence)
    loss = -ELBO
```

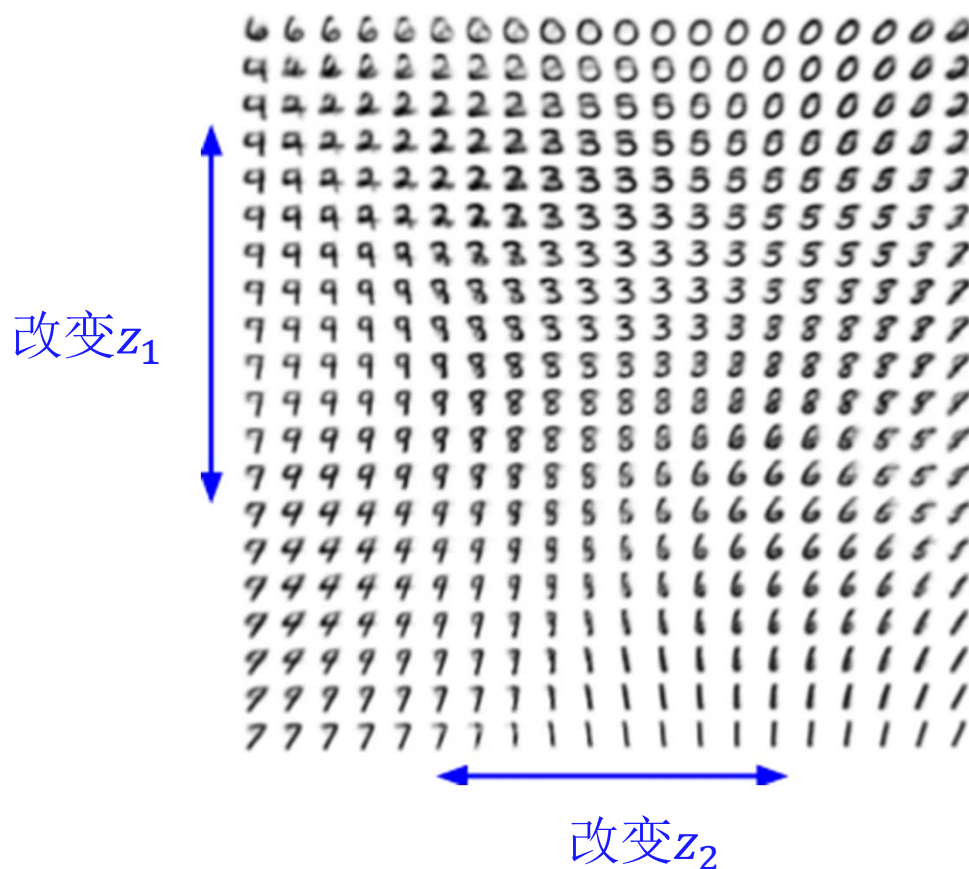
■ 数据生成

- 从先验分布中随机抽取 $\mathbf{z}$ 送入Decoder中，则可随机生成相应的图像



■ 数据生成

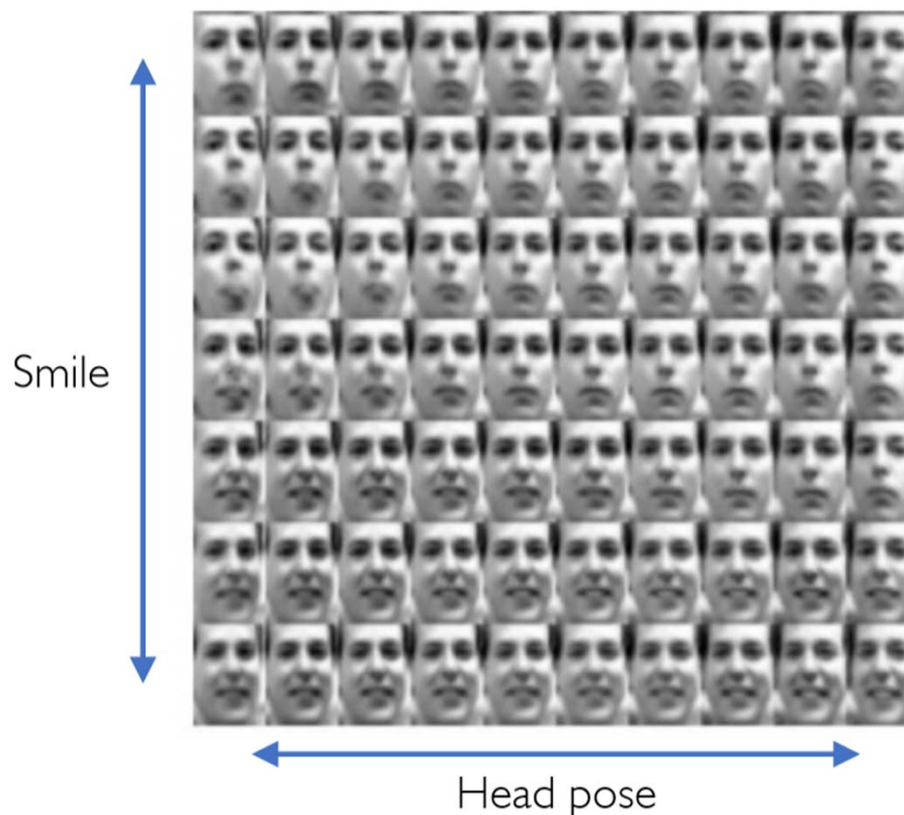
- 例：在手写数字数据集上训练，且隐含变量 $\mathbf{z} = [z_1, z_2]^T$ 为二维向量



■ 数据生成

➤ Disentangled VAE

- 在隐含变量中，每个点之间彼此无关，各自学习输入数据中不同隐含特征
- 对 $\mathbf{z}$ 增加约束，使每一维的隐含分布**相互独立**



■ 数据生成

➤ Disentangled AVE

- 在隐含变量中，每个点之间彼此无关，各自学习输入数据中不同隐含特征
- 对 $\mathbf{z}$ 增加约束，使每一维的隐含分布**相互独立**



Head pose

对隐含变量的某一维度进行平滑扰动，观察相应所生成的数据，来理解该维度隐变量的物理含义

■ 小结

- 自编码器 + 概率分布估计 => 生成模型，可生成新的数据
- 隐含变量是对可观测数据的一种压缩表征
- 通过重构的方式进行无监督学习，降低对数据标注的要求
- 隐含变量的各维度对应着可解释的物理含义

Variational
AutoEncoder

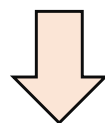


PART THREE

生成对抗网络

Generative Adversarial Network

- 变分自编码器：显式密度估计
 - 对样本分布进行显式建模，并优化求解模型参数
- 只想关注于生成新样本？
 - 不显式地建模数据分布，但直接生成符合数据分布的样本



生成对抗网络
(Generative Adversarial Network, GAN)

借助博弈 (Game play) 思想：通过“两方博弈”，从训练数据分布中学习生成新数据

➤ 问题描述

- 从复杂的高维数据分布空间进行采样，生成新样本

没有直接可行的方法！

➤ 解决思路

- 从一个容易获取的简单分布（如随机噪声）中进行采样，并学习该分布与真实训练数据分布之间的转换

Q: 借助什么来表征该复杂的转换？

A: 神经网络！

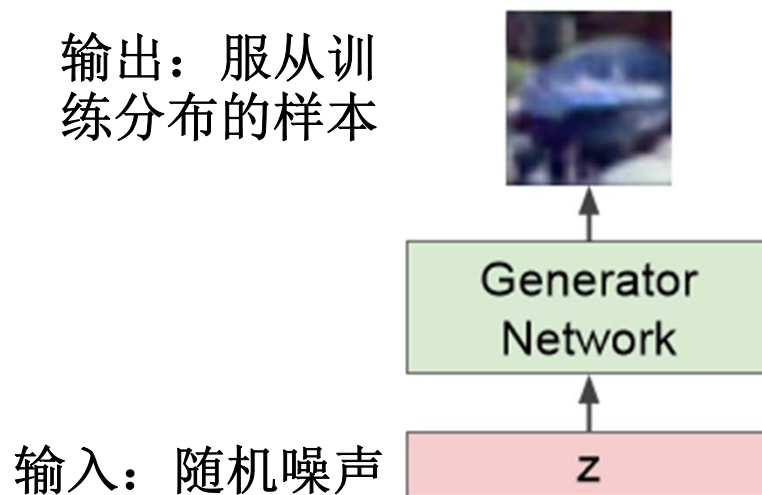
➤ 问题描述

- 从复杂的高维数据分布空间进行采样，生成新样本

没有直接可行的方法!

➤ 解决思路

- 从一个容易获取的简单分布（如随机噪声）中进行采样，并学习该分布与真实训练数据分布之间的转换



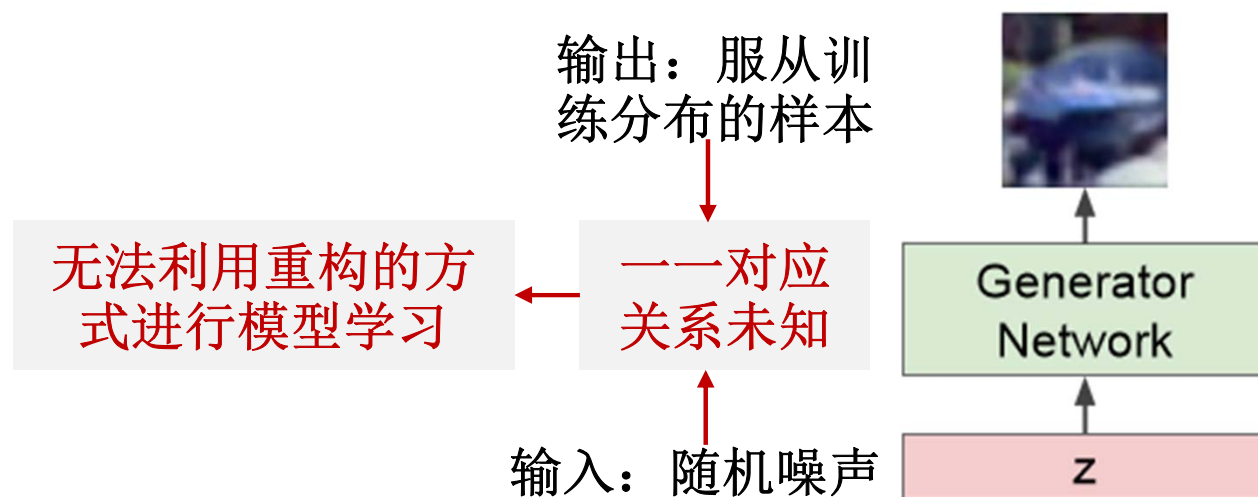
➤ 问题描述

- 从复杂的高维数据分布空间进行采样，生成新样本

没有直接可行的方法!

➤ 解决思路

- 从一个容易获取的简单分布（如随机噪声）中进行采样，并学习该分布与真实训练数据分布之间的转换



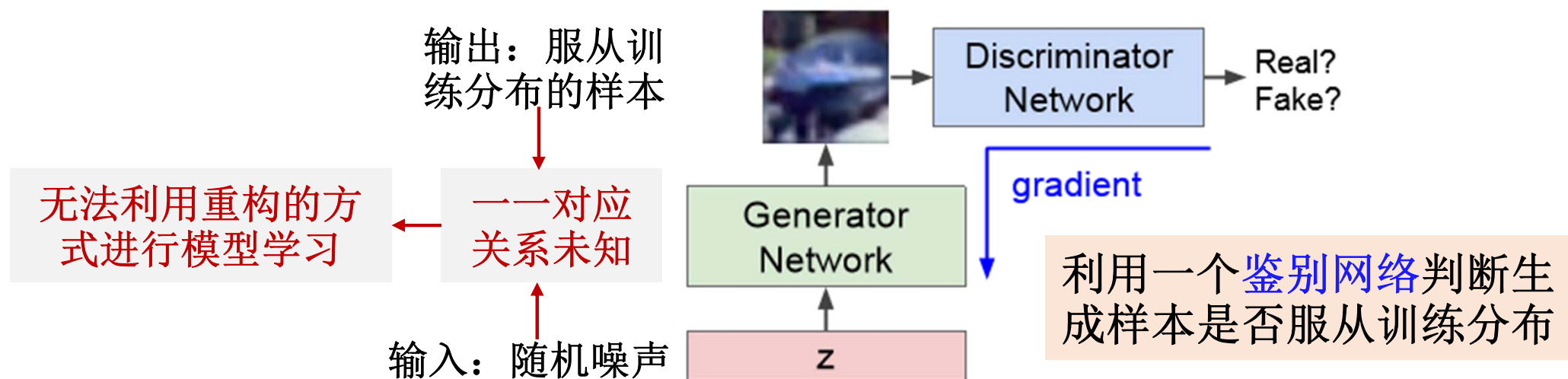
➤ 问题描述

- 从复杂的高维数据分布空间进行采样，生成新样本

没有直接可行的方法!

➤ 解决思路

- 从一个容易获取的简单分布（如随机噪声）中进行采样，并学习该分布与真实训练数据分布之间的转换



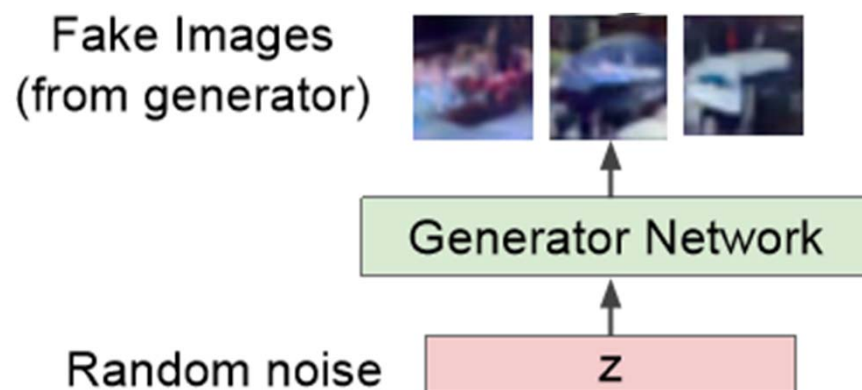
Goodfellow, et al., Generative adversarial nets. NIPS2014.

➤ 由Ian Goodfellow 2014年提出，包括两个网络

- 生成网络 (Generator)

--制造假数据

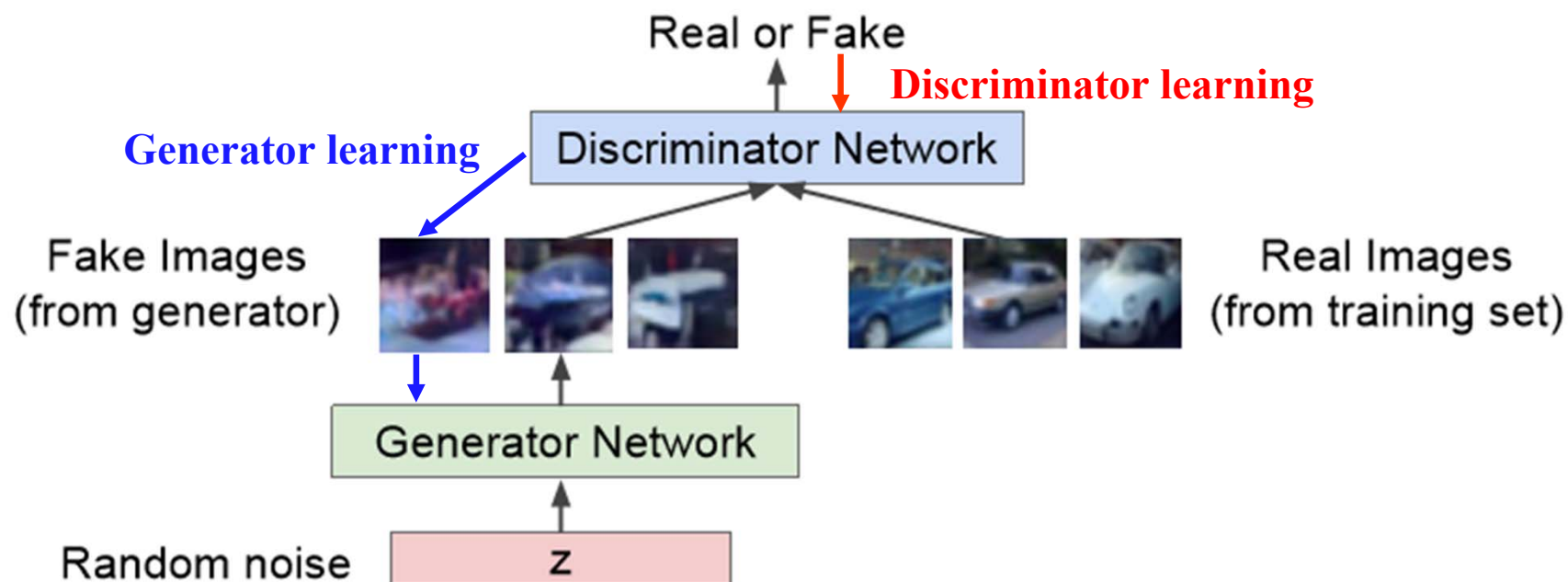
- 鉴别网络 (Discriminator)



Goodfellow, et al., Generative adversarial nets. NIPS2014.

➤ 由Ian Goodfellow 2014年提出，包括两个网络

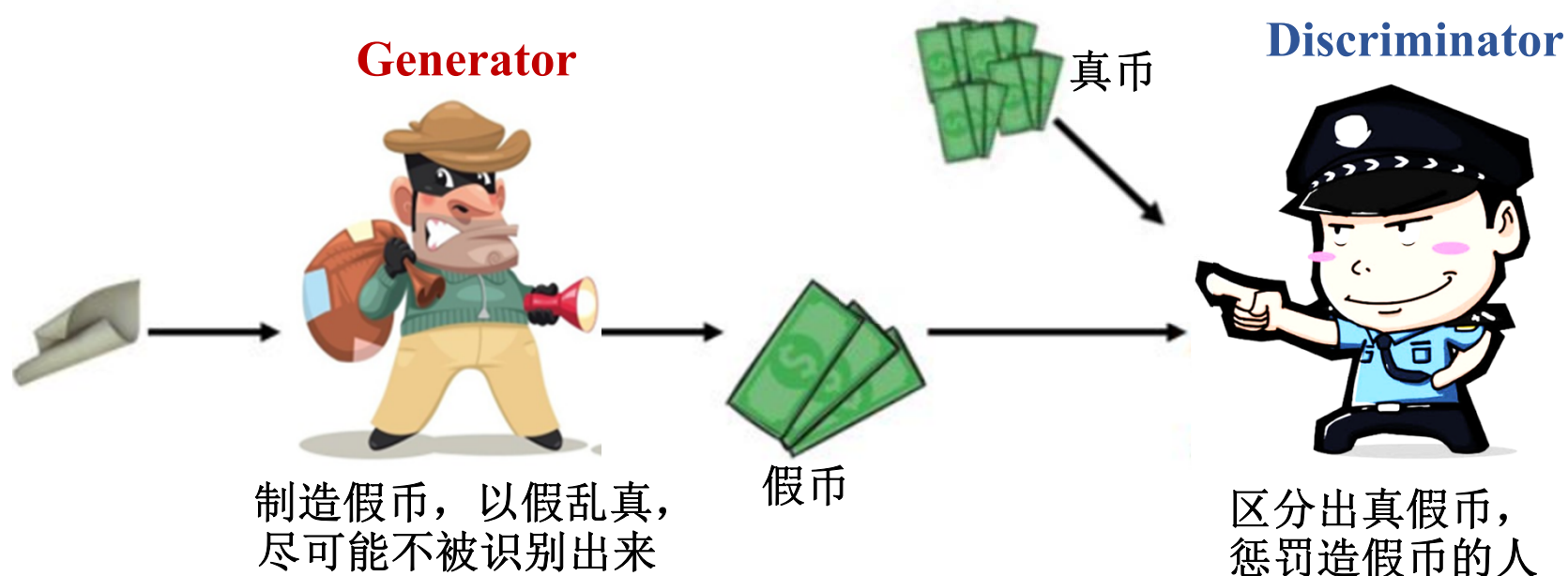
- 生成网络 (Generator)  博弈抗衡  • 鉴别网络 (Discriminator)
- 制造假数据
- 对真实和伪造数据进行鉴别
- 尽可能地模仿训练数据，迷惑鉴别网络
- 发现假数据，惩罚生成网络



Goodfellow, et al., Generative adversarial nets. NIPS2014.

➤ 由Ian Goodfellow 2014年提出，包括两个网络

- 生成网络 (Generator) → 博弈抗衡 ← • 鉴别网络 (Discriminator)
- 制造假数据
- 对真实和伪造数据进行鉴别
- 尽可能地模仿训练数据，迷惑鉴别网络
- 发现假数据，惩罚生成网络



➤ GAN的学习过程: **Minimax Game**

- 鉴别网络 (**Discriminator**): 试图区分真伪数据
- 生成网络 (**Generator**): 试图用制造的假数据来迷惑鉴别网络

Minimax 目标函数:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Generator **Discriminator**

➤ GAN的学习过程: Minimax Game

- 鉴别网络 (Discriminator): 试图区分真伪数据
- 生成网络 (Generator): 试图用制造的假数据来迷惑鉴别网络

对数据真实程度的概率估计, 输出值范围[0, 1]

Minimax 目标函数:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \underbrace{\log D_{\theta_d}(x)}_{\text{对真实数据 } x \text{ 的判断}} + \mathbb{E}_{z \sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\text{对假数据 } G_{\theta_g}(z) \text{ 的判断}}) \right]$$

对真实数据 x 的判断

对假数据 $G_{\theta_g}(z)$ 的判断

- **Discriminator** 试图 **最大化** 目标函数, 使 $D_{\theta_d}(x)$ 接近于1 (真实数据), $D_{\theta_d}(G_{\theta_g}(z))$ 接近于0 (假数据)
- **Generator** 试图 **最小化** 目标函数, 使 $D_{\theta_d}(G_{\theta_g}(z))$ 接近于1 (**Discriminator** 被欺骗, 认为生成数据 $G_{\theta_g}(z)$ 是真实的)

➤ Minimax 学习

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

两个网络交替优化:

- **Discriminator:** 梯度上升

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

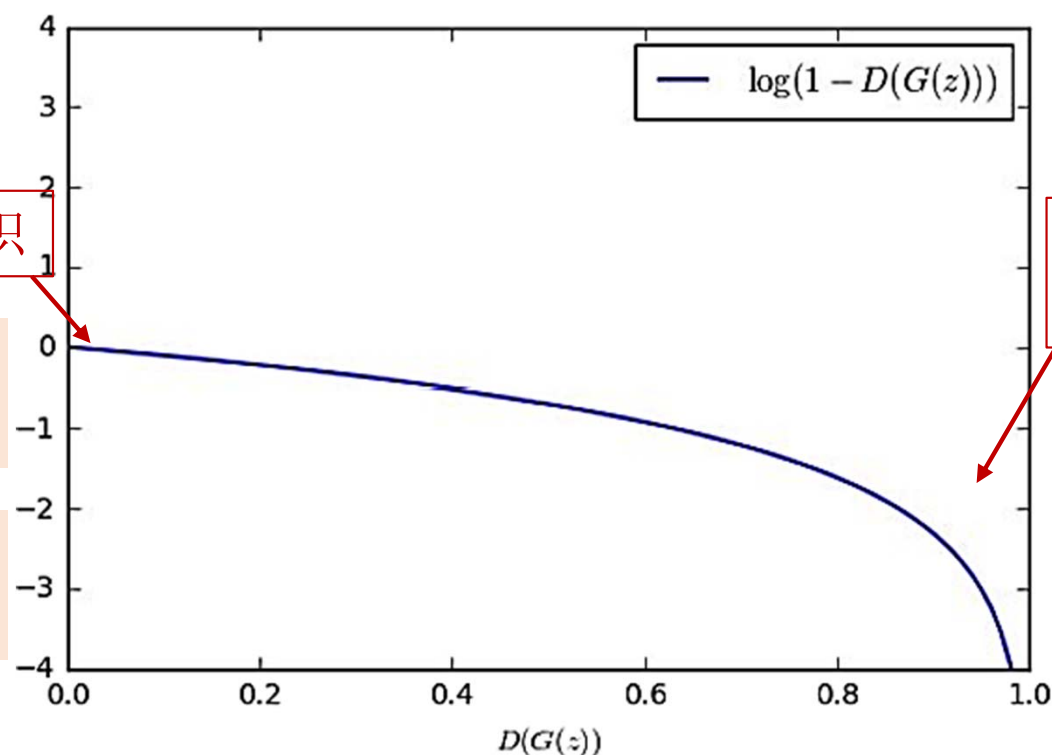
- **Generator:** 梯度下降

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

实际中发现，优化该目标函数的效果并不好！

➤ Minimax 学习

- **Generator** : 梯度下降 $\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$



生成的假数据被轻易识

需要重点学习优化，
以提升Generator

但该区域的梯度相
对较为平坦！

生成数据的质量较
好，被当做真数据

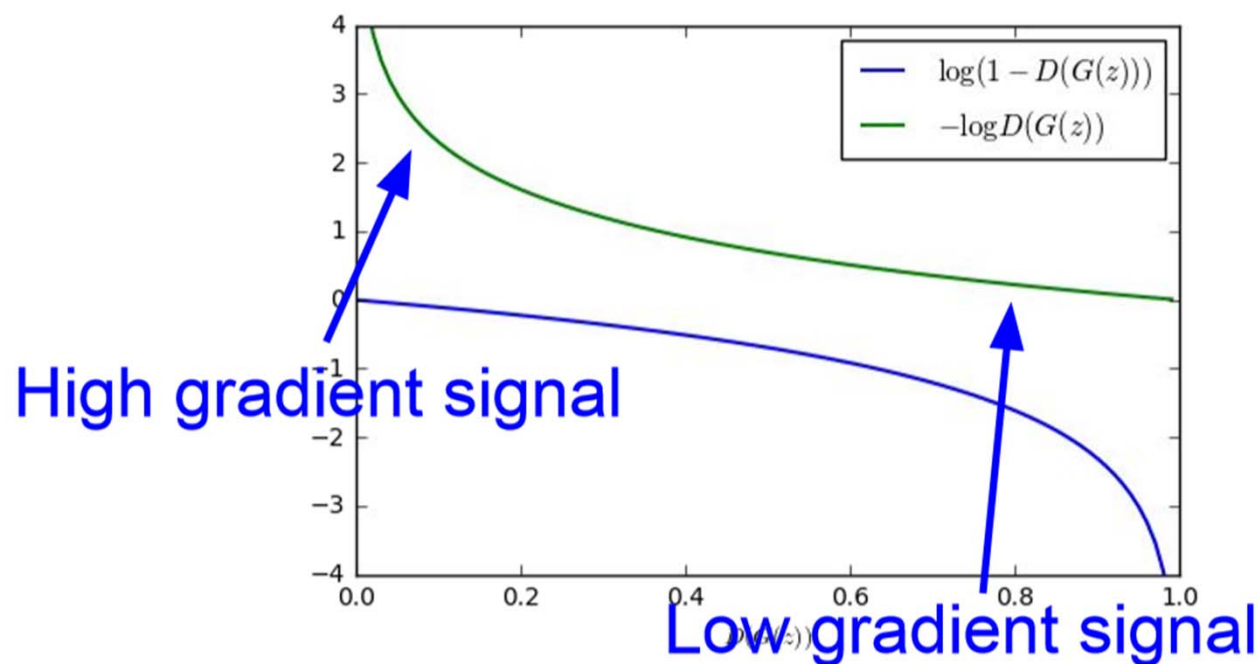
梯度信号被该区域
占主导！

➤ Minimax 学习

- **Generator** : 梯度下降 $\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$

↓ 替换

- **Generator** : 梯度上升 $\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$



➤ Minimax 学习

两个网络交替优化:

- **Discriminator:** 梯度上升

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

- **Generator:** 梯度上升

$$\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$$

相较之前，性能有所改善！

➤ GAN训练整体流程

for 训练总迭代步数 **do**

for Discriminator优化步数 k **do**

- 固定Generator的参数，对Discriminator 先优化 k 步

end for

- 固定Discriminator的参数，优化Generator的参数

end for

➤ GAN训练整体流程

for 训练总迭代步数 **do**

有些情况发现 $k=1$ 较稳定, 有时 $k>1$ 较好, 没有严格的准则

for Discriminator优化步数 k **do**

- 从噪声先验 $p_g(\mathbf{z})$ 中取样minibatch= m 的噪声数据 $\{\mathbf{z}^{(1)}, \dots \mathbf{z}^{(m)}\}$
- 从分布 $p_{data}(x)$ 的训练数据中取样minibatch= m 的真实数据 $\{\mathbf{x}^{(1)}, \dots \mathbf{x}^{(m)}\}$
- 通过随机梯度上升法, 优化更新Discriminator的参数 θ_d :

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D_{\theta_d}(\mathbf{x}^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(\mathbf{z}^{(i)}))) \right]$$

参数固定

end for

- 从噪声先验 $p_g(\mathbf{z})$ 中取样minibatch= m 的噪声数据 $\{\mathbf{z}^{(1)}, \dots \mathbf{z}^{(m)}\}$
- 通过随机梯度上升法, 优化更新Generator的参数 θ_g :

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(D_{\theta_d}(G_{\theta_g}(\mathbf{z}^{(i)})))$$

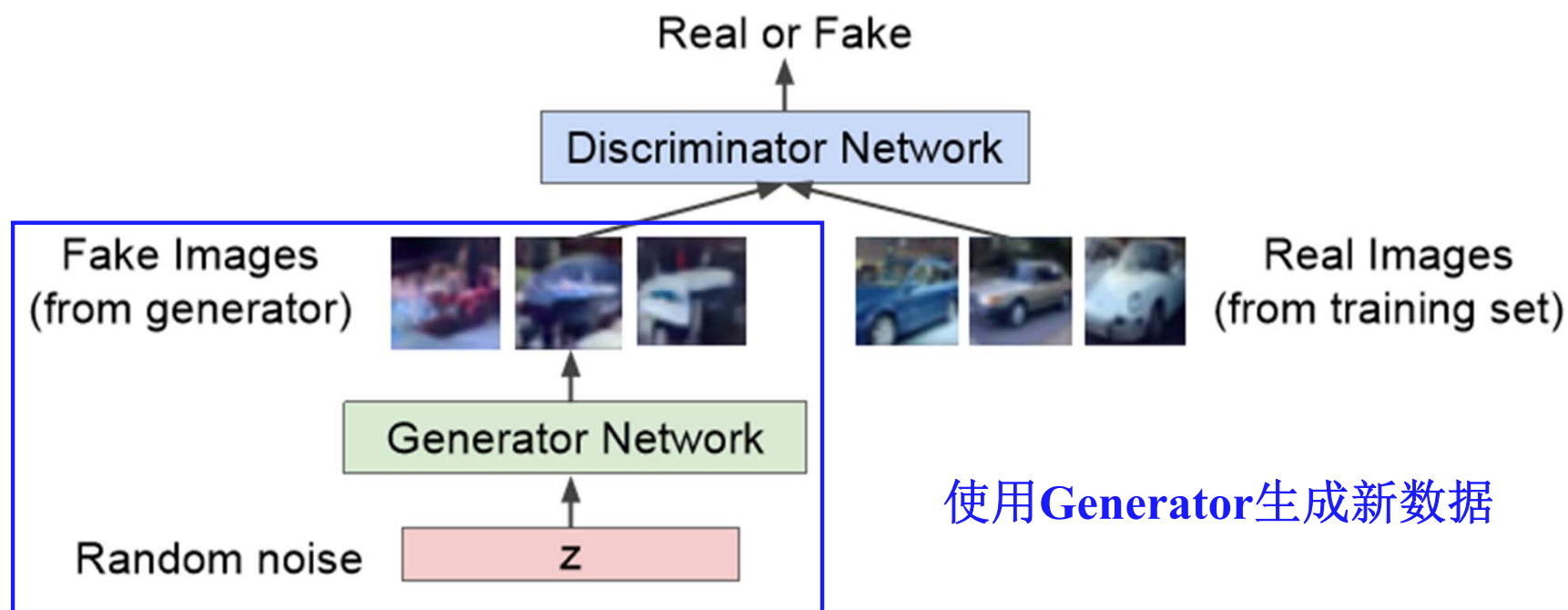
end for

参数固定

Goodfellow, et al., Generative adversarial nets. NIPS2014.

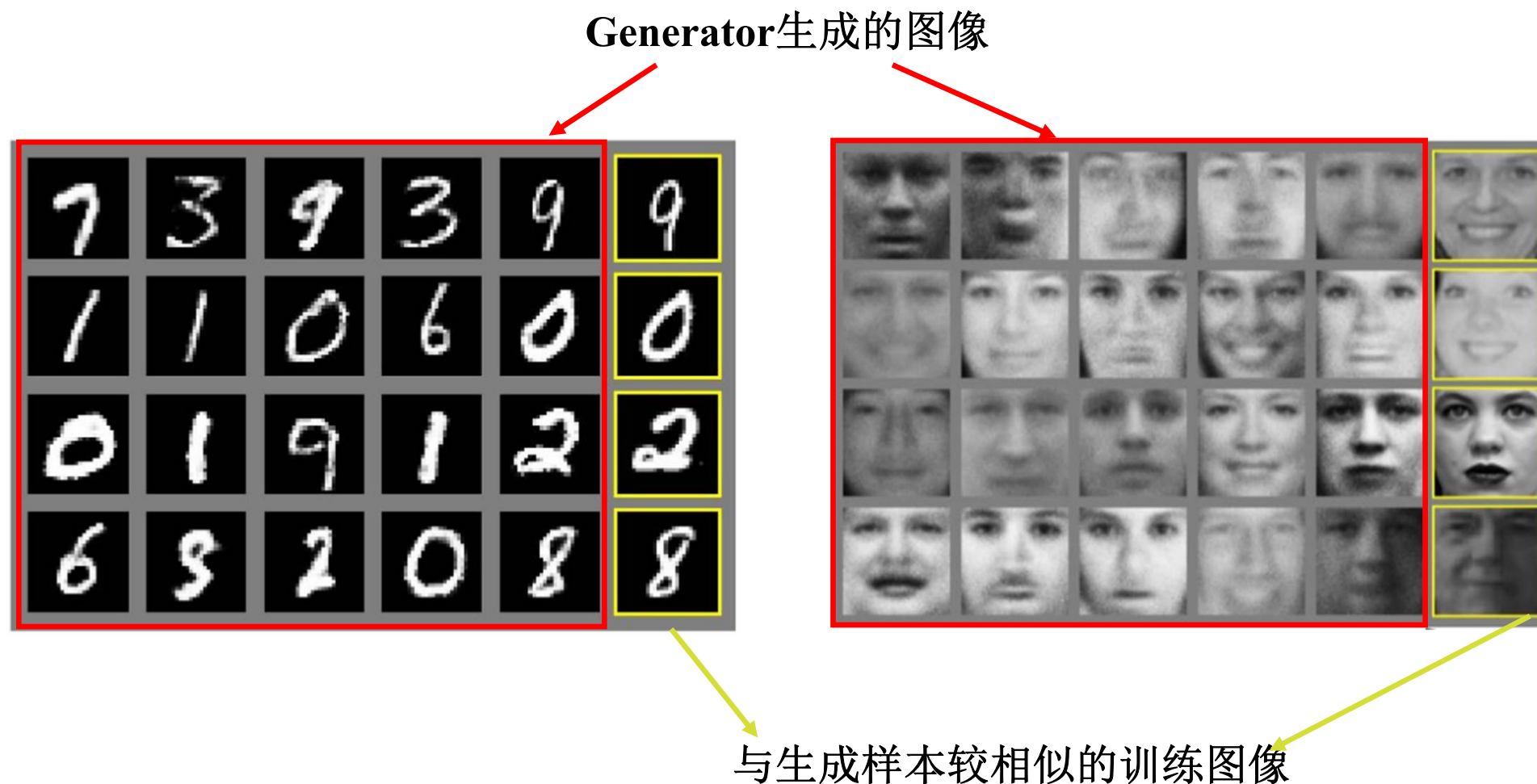
➤ GAN: 双方博弈游戏

- 训练完成后, **Generator**生成的数据足够逼真, 成功迷惑**Discriminator**



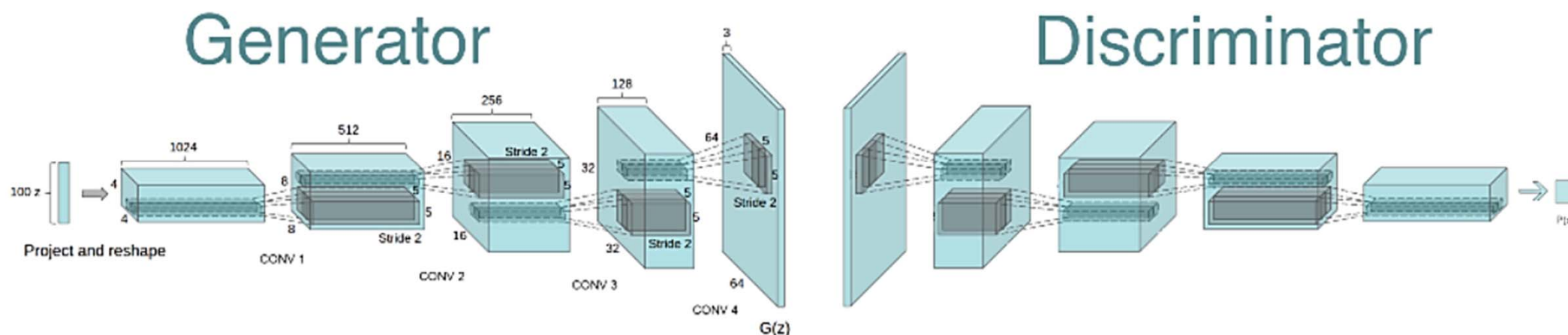
Goodfellow, et al., Generative adversarial nets. NIPS2014.

- 生成样本示例



➤ 深度卷积结构-DCGAN

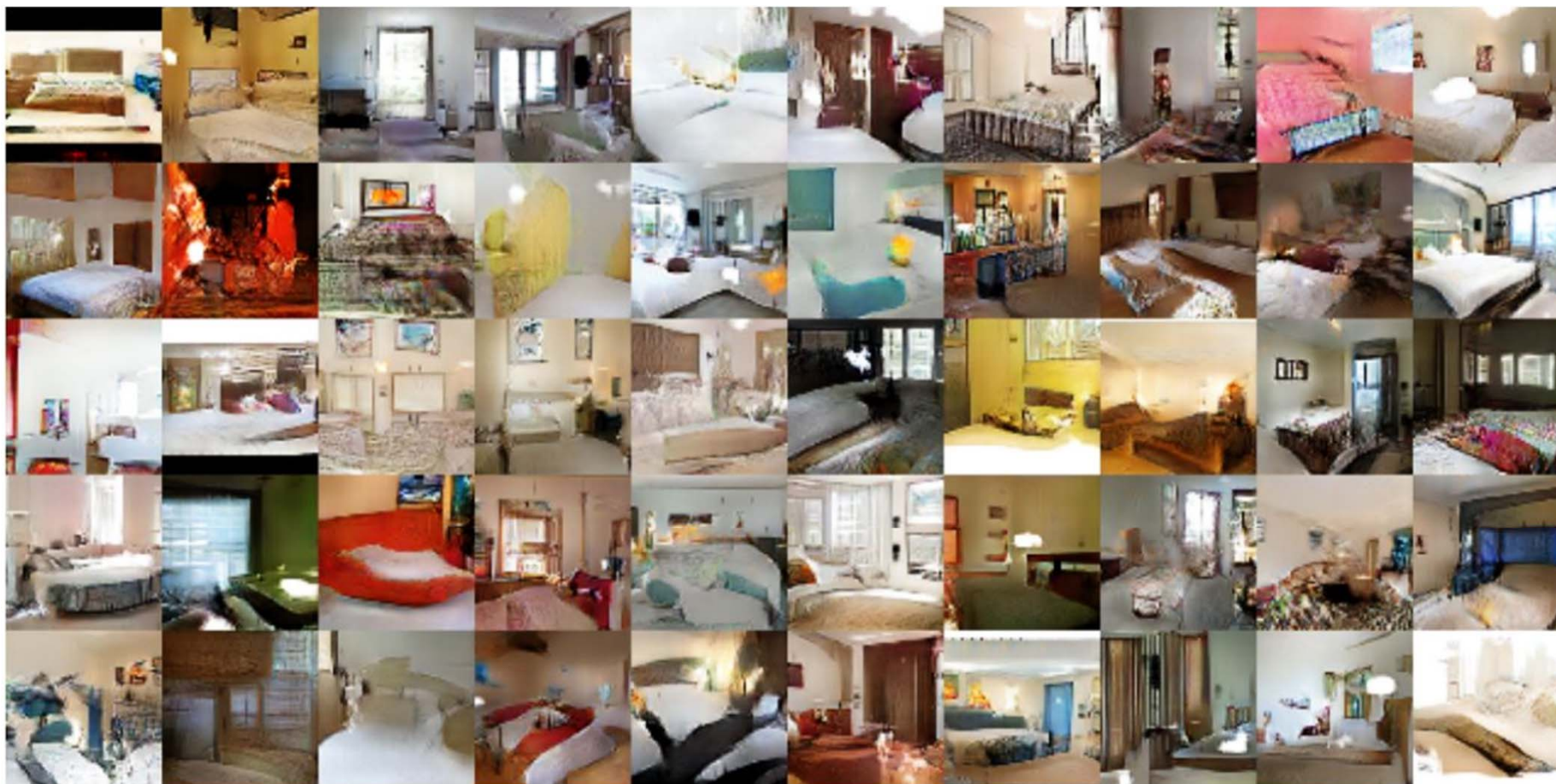
- Generator和Discriminator的池化层均替换为带步长的卷积层和反卷积层
- 两个网络均加入batchnorm层，去掉全连接层



Radford *et al.*, Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks, ICLR 2016.

Radford *et al.*, Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks, ICLR 2016.

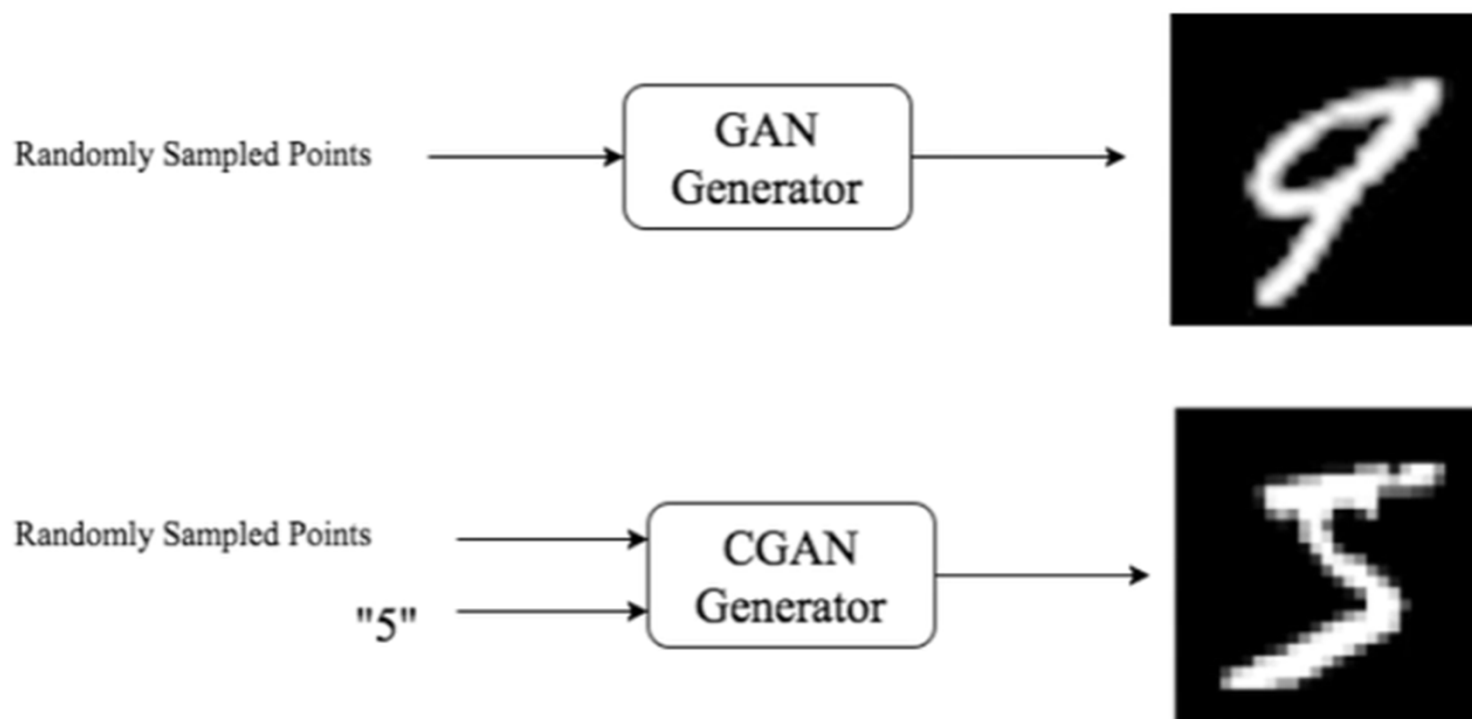
➤ 深度卷积结构-DCGAN



生成的样本看起来更清晰真实！

➤ Conditional GAN

- 输入一个条件 (例如, 图像标签), 来指定想要生成的图像类型



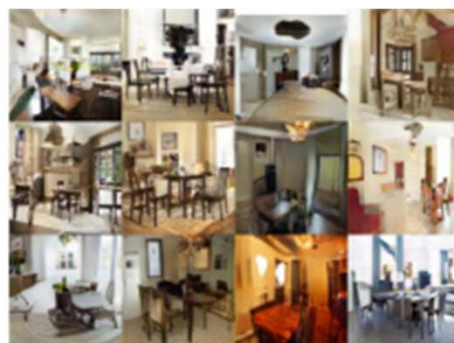
Mirza *et al.*, Conditional Generative Adversarial Nets.

➤ 2017: Explosion of GANs

- Better training and generation



(a) Church outdoor.



(b) Dining room.



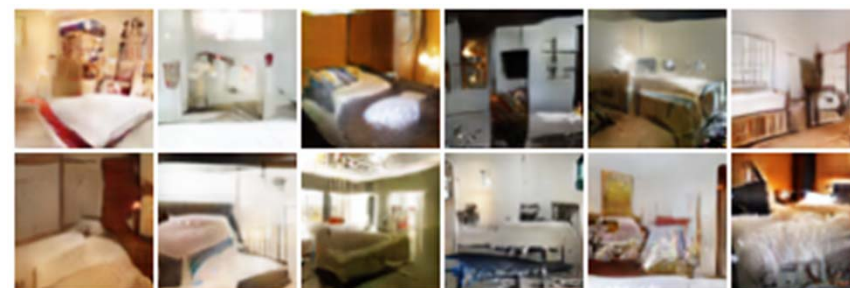
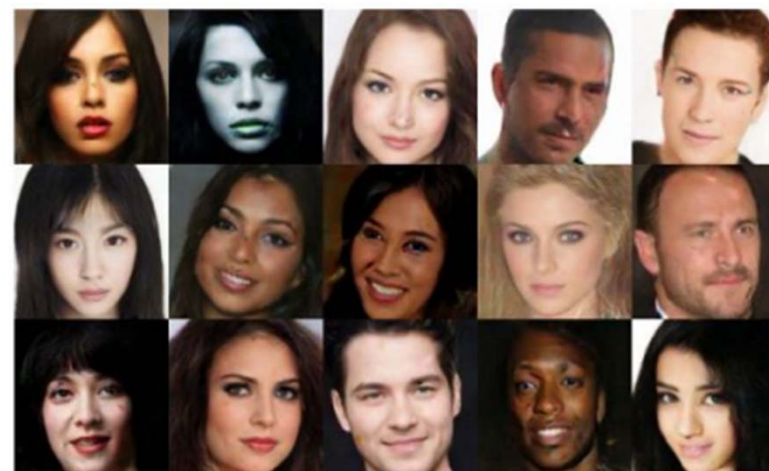
(c) Kitchen.



(d) Conference room.

LSGAN. Mao *et al.* 2017.

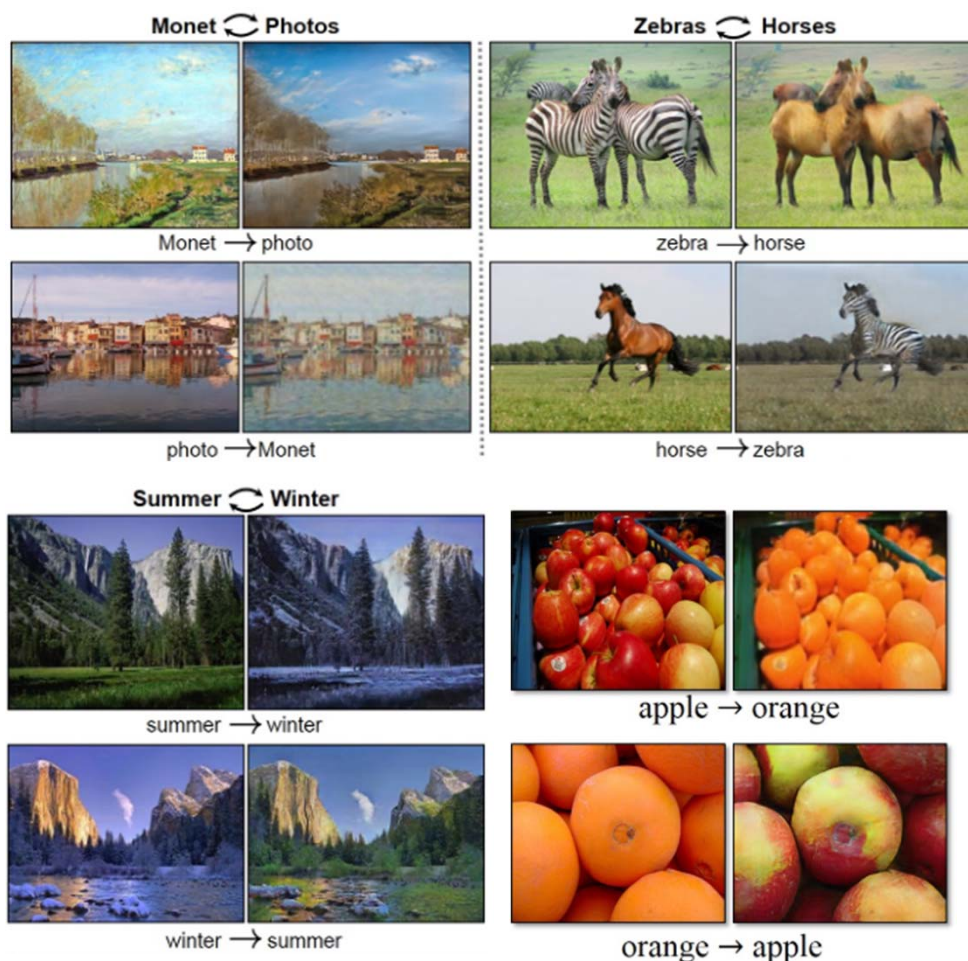
BEGAN. Bertholet *et al.* 2017.



Wasserstein GAN. Arjovsky *et al.* 2017.

➤ 2017: Explosion of GANs

- Source->Target domain transfer



CycleGAN. Zhu *et al.* 2017.

- Text -> Image Synthesis

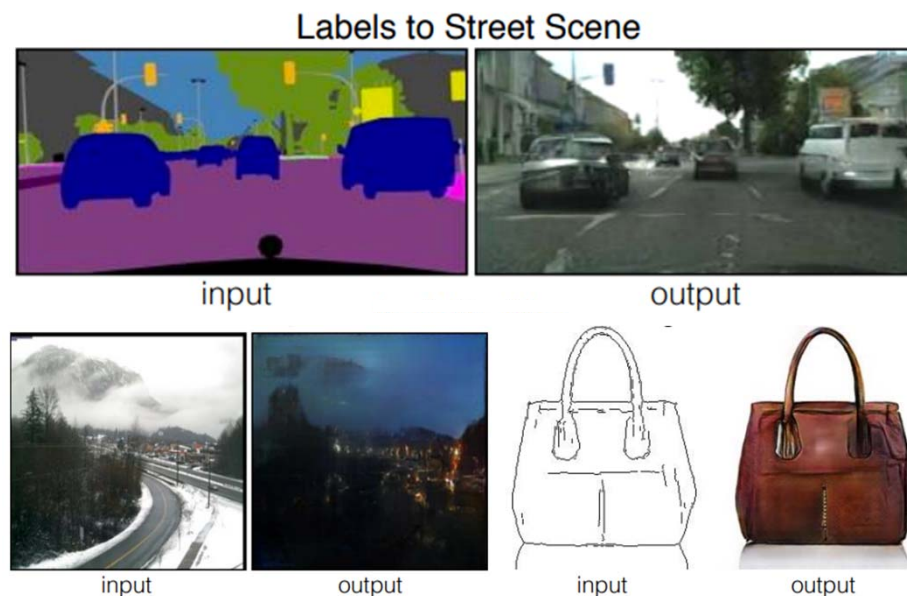
A small bird with a black head and wings and features grey wings

This bird is completely red with black wings and pointy beak



StackGAN. Zhang *et al.* 2017.

- Text -> Image Synthesis



pix2pix. Isola *et al.* 2017.

➤ Other awesome works

Progressive growing of GANs: results



Karras et al., ICLR 2018.

➤ Other awesome works

Style-based generator: results



Karras et al., Arxiv 2018.

➤ Other awesome works

Style-based transfer: results



Karras et al., Arxiv 2018.

■ 小结

- 关注于数据生成问题，**不涉及对数据分布的显式建模问题**
- 借助**博弈思想**：通过“两方博弈”，从训练数据分布中学习生成新数据
- **优点与不足**：
 - 优：能生成非常生动清晰的样本
 - 缺：训练过程不易稳定，需要更多的训练技巧
- **活跃研究领域**：
 - 寻求更好的损失函数，更稳定的训练策略 (Wasserstein GAN, LSGAN等)
 - 广泛的应用 (图像转换、动漫生成、语音识别、去雾等)

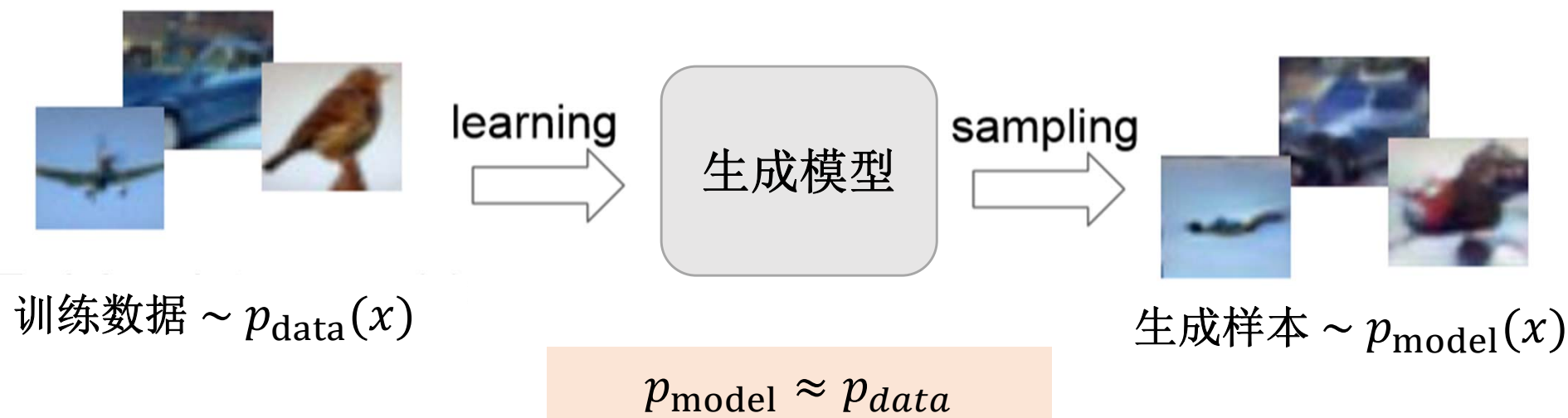
PART FOUR

小结和资源

Take Home Message and Resource

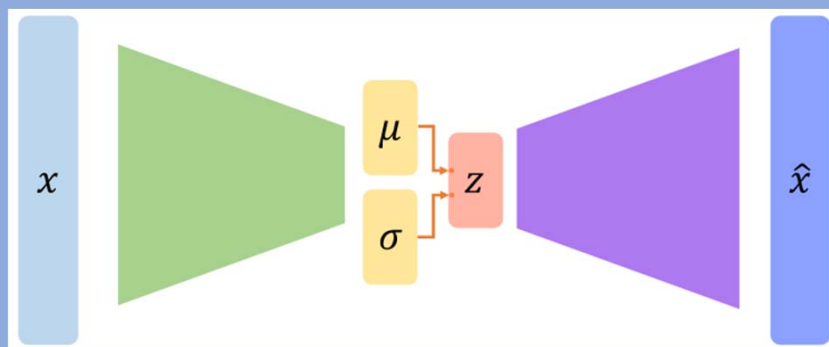
■ 生成网络

- 希望模型生成的新数据与训练数据的真实分布尽可能地相似

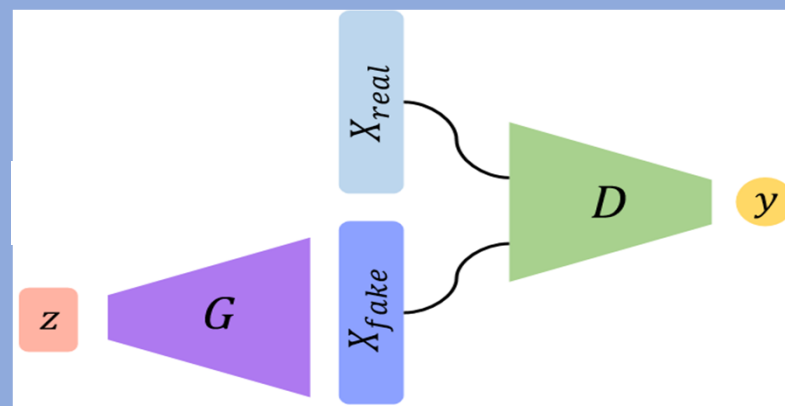


■ 两种生成模型

变分自编码器 (Variational Autoencoder, AVE)



生成对抗网络 (Generative Adversarial Network, GAN)

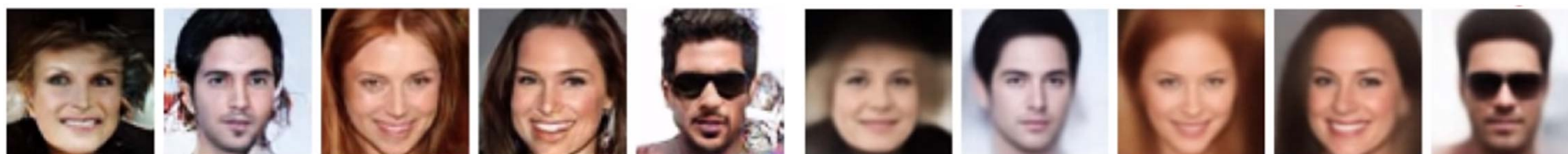


■ 变分自编码器 (VAE)

普通的自编码器 (Normal AutoEncoder)	变分自编码器 (Variational AutoEncoder)
• 作用 -- 学习输入数据的隐含表征 • 只能重构数据, 无法生成新数据 • 优化: -- min 重构误差	• 作用 -- 学习生成新的可观测数据 • 优化: -- min (重构误差+隐变量的约束项) -- 隐变量从正态分布中随机抽取

■ 变分自编码器 (VAE)

生成对抗网络 (Generative Adversarial Network)	变分自编码器 (Variational AutoEncoder)
- 隐式密度估计 - 网络结构: -- Generator + Discriminator - 如何学习: -- Minmax Game 交替训练	- 显式密度估计 - 网络结构: -- Encoder + Decoder - 如何学习: -- min (重构误差+隐变量的约束项)



GAN生成图像的质量更清晰!

➤ Paper

- [1] Karras et al., A Style-Based Generator Architecture for Generative Adversarial Networks, CVPR2019.
- [2] Goodfellow, *et al.*, Generative adversarial nets. NIPS2014.
- [3] Ledig *et al.*, Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network. CVPR2017.
- [4] Isola *et al.*, Image-to-Image Translation with Conditional Adversarial Networks. CVPR2017.
- [5] C Doersch, Tutorial on Variational Autoencoders. arXiv abs/1606.05908.
- [6] Higgins *et al.*, β -VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. ICLR2017.
- [7] Radford et al., Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks, ICLR 2016.

➤ Other useful links ZOO

<https://github.com/hindupuravinash/the-gan-zoo> GAN ZOO

<https://github.com/soumith/ganhacks> Tips for Training GANs